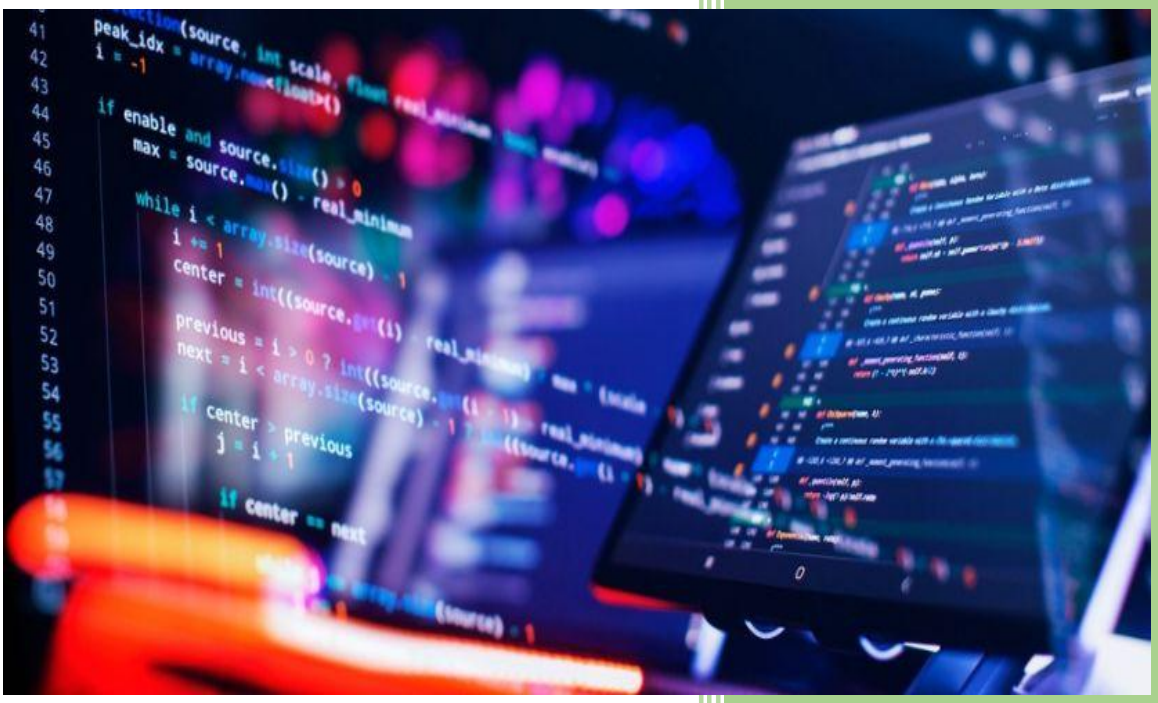
2026

SYSTEME DE RECOMMANDATION



Travail réalisé par :

- Aicha Rihan Ali
- Abass Moussa Djibril
- Abdichakour Ahmed Mohamed
- Abdinasser Robleh Abdillahi

Encadrant : Dr. Moubarek Barre

Sommaire

Remerciement 3
Dédicaces 3
Introduction 4
I. Analyse des besoins 6
A. Fonctionnalités du système 6
1. Besoins fonctionnels 6
2. Besoins non fonctionnels 8
B. Collecte des besoins 9
C. Cahier de charge 9
1. Présentation du projet 10
2. Les contraintes du projet 12
3. Répartition des rôles au sein de l'équipe de projet 13
4. Outils et environnement technique 14
5. Planification du projet 15
6. Livrables attendus 16
II. Planification du projet 18
A. Méthodologie Scrum 18
1. Contexte du projet 18
2. Formation de l'équipe Scrum 18
3. Création du Product Backlog 19
4. Sprint Planning (Planification du Sprint) 19
5. Développement du Sprint 20
6. Daily Scrum 21
7. Sprint Review 21
8. Sprint Retrospective — Rétrospective 21
9. Livraison progressive du système 22
B. Estimations des tâches 22
C. Découpage WBS 23
D. Diagramme de Gantt 24
E. Gestion des risques 25

III. Modélisation du système 25
A. Architecture 25
1. Architecture en couches 26
2. Architecture client-serveur 26
3. Architecture micro-service 27
B. Conception UML 28

1. Diagrammes statiques 28
2. Diagrammes dynamiques 33
IV. Les algorithmes de recommandation 40
A. Filtrage collaboratif 40
B. Filtrage basé sur le contenu 40
C. Approche hybride 41
V. Implémentation 41
A. Environnement de développement 42
1. Langages de programmation utilisés 42
2. Outils et frameworks 43
B. Architecture du code 46
1. Structure des modules 46
2. Communication entre modules (flux UI → API → DB) 46
3. Patterns de conception appliqués 47
VI. Test et validation 48
A. Stratégie de test 48
1. Tests Unitaires 50
2. Tests d'intégration 51
3. Tests fonctionnels 52
B. Scénarios de test 53
C. Résultats et analyse 54
VII. Déploiement et maintenance 55
A. Environnement de déploiement 55
1. Infrastructure et hébergement 55
2. Configuration du serveur 56
B. Procédure de déploiement 56
1. Pipeline CI/CD 57
2. Étapes de mise en production 57
C. Maintenance et évolutivité 58
1. Surveillance et monitoring 58
2. Mises à jour et améliorations 58
Conclusion 60
Bibliographie 61
Annexes 61
Cahier de bord 61

Remerciement

Nous entamons ce projet en exprimant avant tout notre profonde gratitude envers **Dieu**, pour nous avoir accordé la force, la patience et la détermination nécessaires afin de mener à bien ce travail dans de bonnes conditions.
Nous adressons nos sincères remerciements à **l'Université de Djibouti** pour nous avoir offert un cadre académique favorable à l'apprentissage et à l'approfondissement de nos connaissances, ainsi que l'opportunité de réaliser ce projet technique enrichissant.
Nous tenons à exprimer notre reconnaissance particulière à **Monsieur Moubarek**, notre encadrant pédagogique, pour son accompagnement, ses conseils précieux, sa disponibilité et son encadrement rigoureux tout au long du projet de **Système de recommandation**. Son expertise et son sens pédagogique ont grandement contribué à la réussite de ce travail.
Nous remercions également toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet, par leurs conseils, leurs encouragements ou leur soutien moral.
Enfin, nous exprimons notre profonde gratitude à nos familles pour leur soutien inconditionnel, leur patience et leurs encouragements constants tout au long de notre parcours académique.

Dédicaces

Nous dédions ce travail à nos familles, pour leur soutien inconditionnel, leur patience et leurs encouragements constants tout au long de notre parcours académique.

Nous dédions également ce projet à l'Université de Djibouti, qui nous a offert un cadre d'apprentissage favorable au développement de nos connaissances et compétences dans le domaine de l'informatique et du génie logiciel.

Une dédicace particulière est adressée à Monsieur **Moubarek** pour son encadrement, ses conseils précieux et son accompagnement tout au long de la réalisation de notre projet portant sur le développement d'un système de recommandation intégré à une plateforme e-commerce.

Nous dédions enfin ce travail à nos camarades de la filière pour leur esprit de collaboration, leur entraide et la motivation partagée, qui ont contribué à créer un environnement d'apprentissage dynamique et enrichissant.

Introduction

Ces dernières années, le commerce électronique a pris une place de plus en plus importante dans la vie quotidienne. Les plateformes en ligne proposent aujourd'hui des catalogues immenses, parfois des millions de produits, ce qui rend la navigation difficile pour les utilisateurs. Face à cette surcharge d'informations, trouver rapidement ce qu'on cherche, ou découvrir un produit qu'on aurait aimé, devient un vrai défi. C'est dans ce contexte que les systèmes de recommandation sont apparus comme une réponse concrète et efficace à ce problème.

Ces systèmes permettent de personnaliser l'expérience de chaque utilisateur en lui proposant des produits adaptés à ses goûts et à ses habitudes d'achat. Des plateformes comme Amazon ou Alibaba utilisent ce type de technologie depuis longtemps, et leurs résultats montrent à quel point la recommandation peut améliorer la satisfaction des utilisateurs et augmenter les ventes. Ce n'est donc pas étonnant que le sujet attire autant d'attention, aussi bien dans le monde professionnel que dans la recherche.

C'est dans ce cadre que se pose la problématique centrale de ce projet : **comment concevoir un système de recommandation capable de proposer des produits pertinents à chaque utilisateur, en tenant compte de ses comportements et de ses préférences, tout en restant techniquement réalisable et performant ?** Cette question soulève des enjeux à la fois algorithmiques, liés au choix et à l'implémentation des méthodes de recommandation, et pratiques, liés à la qualité des données et à la pertinence des résultats produits.

Cependant, concevoir un système de recommandation efficace n'est pas une tâche simple. Plusieurs approches existent : le filtrage collaboratif, qui se base sur les comportements d'autres utilisateurs similaires, le filtrage basé sur le contenu, qui analyse les caractéristiques des produits, et les approches hybrides qui combinent les deux. Chacune a ses avantages et ses limites, et le choix entre elles dépend largement du contexte et des données disponibles. C'est autour de cette question que s'articule le présent projet.

L'objectif de ce travail est de concevoir et d'implémenter un système de recommandation de produits dans un environnement e-commerce. Cela implique non seulement un travail technique sur les algorithmes et le développement, mais aussi une démarche méthodologique

complète : analyse des besoins, planification, modélisation, tests et déploiement. L'idée est de produire une solution réaliste, en appliquant les outils et les méthodes étudiés tout au long de la licence.

Ce rapport présente l'ensemble des étapes de ce projet. Il commence par une analyse des besoins, suivie d'une phase de planification et de modélisation du système. Les algorithmes de recommandation retenus sont ensuite expliqués et justifiés, avant de passer à l'implémentation proprement dite. Les phases de test et de déploiement viennent clôturer le travail technique. Une conclusion générale dresse enfin le bilan du projet et propose quelques pistes d'amélioration pour la suite.

I. Analyse des besoins

A. Fonctionnalités du système

L'analyse des fonctionnalités constitue la première étape concrète du projet. Elle consiste à recenser et à formaliser l'ensemble de ce que le système doit faire, et comment il doit le faire, avant d'entrer dans la phase de conception. Dans le cadre de ce projet, les fonctionnalités ont été organisées en deux grandes catégories : les besoins fonctionnels, qui décrivent les services rendus à l'utilisateur, et les besoins non fonctionnels, qui définissent les contraintes de qualité et de performance auxquelles le système doit se conformer.

1. Besoins fonctionnels

a) Gestion des utilisateurs

Le système doit permettre à tout visiteur de créer un compte personnel en renseignant ses informations de base (nom, adresse e-mail, mot de passe). Une fois inscrit, l'utilisateur peut se connecter de manière sécurisée et accéder à son espace personnel, depuis lequel il peut consulter et modifier son profil, suivre ses commandes et visualiser ses recommandations personnalisées.

La gestion des sessions doit garantir qu'un utilisateur non connecté ne puisse pas accéder aux fonctionnalités réservées aux membres. Une fonctionnalité de récupération de mot de passe doit également être prévue pour ne pas bloquer l'accès au compte en cas d'oubli.

- Inscription et création de compte avec validation des données saisies.
- Authentification sécurisée par identifiant et mot de passe.
- Consultation et modification du profil utilisateur.
- Gestion des sessions avec déconnexion automatique après inactivité.
- Historique des achats et des produits consultés accessible depuis le profil.

b) Navigation et consultation du catalogue

L'application doit offrir une expérience de navigation claire et efficace au sein du catalogue de produits. Les utilisateurs doivent pouvoir parcourir les produits par catégorie, effectuer des recherches par mots-clés et affiner les résultats à l'aide de filtres (fourchette de prix, catégorie, note moyenne, disponibilité). Chaque produit dispose d'une fiche détaillée présentant sa description, ses caractéristiques, son prix, ses photos et les avis des autres utilisateurs.

- Navigation par catégories et sous-catégories de produits.
- Barre de recherche avec suggestions automatiques.

- Filtres combinables (prix, catégorie, note, disponibilité).
- Fiche produit détaillée avec photos, description, prix et avis.
- Système de notation et de commentaires par les utilisateurs connectés.

c) *Système de recommandation*

C'est la fonctionnalité centrale du projet. Le moteur de recommandation doit être capable de générer des suggestions personnalisées pour chaque utilisateur, en se basant sur son historique de navigation, ses achats passés, ses notes et les comportements d'utilisateurs ayant des goûts similaires. Ces recommandations doivent être affichées de manière dynamique à plusieurs endroits de l'interface : sur la page d'accueil, sur les fiches produites et dans une section dédiée du profil utilisateur.

- Affichage de recommandations personnalisées sur la page d'accueil.
- Suggestions de produits similaires sur chaque fiche produit.
- Section « Les utilisateurs ayant acheté ceci ont aussi acheté... ».
- Mise à jour des recommandations en temps réel après chaque interaction.
- Recommandations de démarrage pour les nouveaux utilisateurs (basées sur le contenu).

d) *Gestion du panier et des commandes*

Le système doit intégrer un module de gestion du panier permettant à l'utilisateur d'ajouter, de modifier ou de supprimer des produits avant de valider sa commande. Bien que le paiement en ligne ne soit pas dans le périmètre de ce projet, le processus de commande doit être simulé de bout en bout pour permettre la collecte des données d'interaction nécessaires au moteur de recommandation.

- Ajout et suppression de produits dans le panier.
- Modification des quantités et visualisation du montant total.
- Validation de commande avec confirmation par e-mail simulée.
- Suivi de l'état des commandes (en cours, expédiée, livrée).

e) *Interface d'administration*

Une interface d'administration distincte de l'interface utilisateur doit être prévue pour permettre à l'administrateur de gérer le catalogue de produits (ajout, modification, suppression), de superviser les comptes utilisateurs et de consulter des statistiques globales sur l'utilisation du système. Cette interface doit être accessible uniquement aux utilisateurs disposant des droits administrateurs.

- Gestion complète du catalogue produits (CRUD).
- Supervision et gestion des comptes utilisateurs.
- Consultation de statistiques : produits les plus consultés, utilisateurs actifs, taux de recommandation.
- Accès restreint par rôle (administrateur uniquement).

2. Besoins non fonctionnels

a) Performance

Le système doit garantir des temps de réponse acceptables pour assurer une navigation fluide. Le chargement des pages de catalogue et des fiches produits doit s'effectuer en moins de deux secondes dans des conditions normales d'utilisation. La génération des recommandations personnalisées, qui implique des calculs algorithmiques plus lourds, doit rester en dessous de trois secondes. Pour atteindre ces objectifs, des mécanismes de mise en cache des résultats de recommandation et d'indexation des collections PostgreSQL seront mis en place.

b) Sécurité

La sécurité des données est une exigence non négociable, en particulier pour les informations personnelles des utilisateurs. Les mots de passe doivent être stockés sous forme hachée avec un algorithme robuste (bcrypt). Les communications entre le client et le serveur doivent être protégées. Le système doit être protégé contre les attaques les plus courantes : injections, cross-site scripting (XSS) et falsification de requêtes (CSRF). L'accès aux routes d'administration doit être strictement contrôlé par un système d'authentification et de gestion des rôles.

c) Disponibilité et fiabilité

L'application doit être accessible en continu, avec un taux de disponibilité élevé. Dans le cadre de ce projet académique, cela se traduit par une architecture robuste capable de gérer des erreurs sans planter notamment grâce à une gestion des exceptions rigoureuse en TypeScript (Node.js / Next.js) et par la mise en place de mécanismes de récupération en cas de perte de connexion à PostgreSQL via Prisma ORM.

d) Compatibilité et ergonomie

L'interface utilisateur développée en HTML et JavaScript doit être compatible avec les navigateurs web les plus répandus, à savoir Chrome, Firefox, Edge et Safari. Elle doit adopter un design responsif pour s'adapter aux différentes tailles d'écran, des ordinateurs de bureau aux smartphones. Sur le plan de l'ergonomie, l'interface doit être intuitive et ne pas nécessiter de formation particulière pour être utilisée, en respectant les conventions de design des plateformes e-commerce actuelles.

e) Maintenabilité et évolutivité

Le code doit être structuré, documenté et organisé de manière modulaire pour faciliter sa maintenance et son évolution future. L'utilisation de bonnes pratiques de développement (conventions de nommage, séparation des responsabilités, commentaires) et d'une architecture en couches claire doit permettre à un développeur extérieur au projet de comprendre et de modifier le code sans difficulté majeure. L'architecture doit également être conçue pour permettre une montée en charge progressive, notamment via l'utilisation de PostgreSQL qui supporte nativement la mise à l'échelle horizontale.

B. Collecte des besoins

La collecte des besoins est une étape méthodologique qui précède la rédaction formelle des spécifications. Elle consiste à rassembler, à analyser et à consolider l'ensemble des informations nécessaires pour définir avec précision ce que le système doit faire. Dans le cadre de ce projet, cette collecte a été réalisée selon plusieurs approches complémentaires.

La première approche a consisté en une analyse des systèmes existants. Des plateformes e-commerce reconnues comme Amazon, Cdiscount ou encore AliAbaba ont été étudiées pour identifier les fonctionnalités standard d'un système de recommandation en production. Cette analyse comparative a permis de dégager les grandes familles de fonctionnalités attendues par les utilisateurs et de s'en inspirer pour définir le périmètre fonctionnel de ce projet.

La deuxième approche a reposé sur une étude de la littérature technique et scientifique relative aux systèmes de recommandation. La lecture d'articles académiques et de documentation technique a permis d'identifier les exigences non fonctionnelles spécifiques à ce type de système, notamment en termes de performance algorithmique, de qualité des données et de scalabilité. Ces éléments ont directement influencé les choix architecturaux et algorithmiques du projet.

Enfin, une réflexion sur les scénarios d'utilisation, ou user stories, a été menée pour se placer du point de vue de l'utilisateur final et s'assurer que les fonctionnalités identifiées répondent bien à des besoins concrets. Par exemple, le scénario d'un nouvel utilisateur qui découvre la plateforme pour la première fois a permis d'identifier la nécessité d'un mécanisme de recommandation de démarrage (cold start) basé sur le contenu, indépendant de tout historique d'interactions.

C. Cahier de charge

1. Présentation du projet

a) Contexte

Le commerce électronique occupe aujourd'hui une place centrale dans les habitudes de consommation. Les plateformes en ligne proposent des catalogues de plus en plus vastes, ce qui rend la recherche de produits pertinents de plus en plus difficile pour les utilisateurs. Face à cette surcharge informationnelle, les systèmes de recommandation se sont imposés comme une solution incontournable pour personnaliser l'expérience d'achat et aider chaque utilisateur à trouver les produits qui correspondent réellement à ses goûts et à ses besoins.

C'est dans ce contexte que s'inscrit ce projet de fin de licence. Il vise à concevoir et à développer un système de recommandation de produits intégré à une plateforme e-commerce, en exploitant des algorithmes d'apprentissage automatique et une architecture web moderne. Ce projet est mené dans le cadre académique d'une formation en informatique, avec pour objectif de mettre en pratique les compétences acquises tout au long du cursus.

b) 1.2 Objectifs

L'objectif principal de ce projet est de concevoir un système capable de proposer des recommandations de produits personnalisées à chaque utilisateur, en se basant sur son historique d'interactions, ses préférences et les comportements d'utilisateurs similaires. Ce système doit s'intégrer dans une application web e-commerce fonctionnelle.

Les objectifs spécifiques du projet sont les suivants :

- Analyser les besoins fonctionnels et non fonctionnels d'un système de recommandation e-commerce.
- Concevoir une architecture logicielle robuste, modulaire et évolutive.
- Implémenter et comparer plusieurs algorithmes de recommandation (filtrage collaboratif, filtrage basé sur le contenu, approche hybride).
- Développer une interface utilisateur intuitive et responsive permettant de visualiser les recommandations générées.
- Valider le système à travers des tests fonctionnels et évaluer la pertinence des recommandations produites.
- Documenter l'ensemble du projet de manière rigoureuse et complète.

c) 1.3 Périmètre du projet

Le périmètre de ce projet couvre la conception et le développement d'une application web e-commerce intégrant un moteur de recommandation. L'application comprend une interface utilisateur permettant la navigation dans un catalogue de produits, la gestion d'un compte personnel, la consultation de recommandations personnalisées et la simulation d'un processus de commande.

Le projet inclut également une interface d'administration permettant la gestion du catalogue produits et la supervision des utilisateurs. En revanche, certains aspects sont volontairement exclus du périmètre afin de rester dans un cadre académique réaliste : le paiement en ligne réel, la logistique et la gestion des stocks en temps réel, ainsi que le déploiement sur une infrastructure cloud à grande échelle. Ces éléments pourraient faire l'objet d'une évolution future du système.

2. Les contraintes du projet

a) Contraintes techniques

Le projet est soumis à plusieurs contraintes techniques qui ont orienté les choix de conception et d'implémentation. La première contrainte concerne l'environnement technologique : les technologies retenues **(**TypeScript, JavaScript, HTML/CSS (Tailwind) et PostgreSQL via Prisma ORM**)** ont été choisies en cohérence avec les compétences disponibles et les outils accessibles dans le cadre académique. Le projet doit fonctionner sur un environnement local sans nécessiter d'infrastructure cloud payante.

La deuxième contrainte technique est liée aux données. Un système de recommandation nécessite un volume de données d'interactions utilisateur-produit suffisant pour produire des résultats pertinents. Dans le cadre de ce projet, les données sont simulées ou générées artificiellement, ce qui constitue une limite réelle sur la qualité des recommandations produites. Le système doit néanmoins être conçu pour fonctionner correctement avec des données réelles en production.

Enfin, les contraintes de performance imposent que les recommandations soient générées en moins de trois secondes et que les pages se chargent en moins de deux secondes, y compris sur des machines aux ressources limitées. Ces exigences influencent directement les choix algorithmiques et les optimisations à mettre en place **(**mise en cache, optimisation des requêtes Prisma et indexation dans PostgreSQL**).**

b) Contraintes humaines et organisationnelles

Ce projet est réalisé dans un cadre académique, ce qui implique des contraintes organisationnelles spécifiques. Le projet est mené par un étudiant seul, ce qui signifie que toutes les phases : analyse, conception, développement, test et documentation, doivent être assurées par la même personne. Cela exige une organisation rigoureuse et une priorisation claire des tâches tout au long du projet.

Par ailleurs, la montée en compétences sur certaines technologies, notamment les algorithmes de machine learning en Python et l'utilisation avancée de PostgreSQl, représente une contrainte humaine réelle. Du temps a été alloué en début de projet pour la formation et le prototypage, afin de réduire les risques liés à ces apprentissages. Enfin, les disponibilités limitées inhérentes au statut étudiant (cours, autres matières, examens) constituent une contrainte organisationnelle à prendre en compte dans la planification.

c) Contraintes temporelles

Le projet s'inscrit dans un calendrier académique contraint. La durée totale allouée est de douze semaines, du lancement du projet jusqu'à la remise du rapport final et la soutenance.

Ce délai impose une gestion rigoureuse du temps et une priorisation des fonctionnalités selon leur importance et leur complexité.

Pour respecter cette contrainte, une approche itérative a été adoptée : les fonctionnalités essentielles (must have) sont développées en priorité, tandis que les fonctionnalités secondaires (should have et could have) sont planifiées en fonction du temps restant. Des jalons intermédiaires ont été définis à la fin de chaque phase pour s'assurer que le projet avance conformément au planning.

3. Répartition des rôles au sein de l'équipe de projet

Rôle	**Responsable**	**Responsabilités principale**
Chef de projet	Abdinasser	Planification, suivi de l'avancement, gestion des risques
Analyste	Ensemble de l'équipe	Collecte et analyse des besoins
Architecte logiciel	Abdichakour	Conception de l'architecture et choix technologiques
Développeur Full-Stack	Abdinasser / Abdichakour	Développement frontend, backend et base de données
Testeur	Abdinasser / Abdichakour	Test et validation du système
Rédacteur	Aicha / Abass	Documentation technique et rédaction du rapport

Encadreur pédagogique	Dr. Moubarek	Supervision, orientation et validation des étapes

- Bien que les rôles et responsabilités aient été définis, le travail a été réalisé de manière collaborative, chaque membre participant à l'ensemble des tâches du projet.

4. Outils et environnement technique

a) Outils logiciels utilisés

Le projet Nexora repose sur un ensemble de technologies modernes permettant de développer une application web performante, scalable et maintenable.

Langages de programmation

– **TypeScript / JavaScript** : utilisés pour le développement full-stack, garantissant flexibilité et typage sécurisé côté serveur et client.
– **HTML / CSS (Tailwind CSS) :** utilisés pour la structuration et la mise en forme de l'interface utilisateur avec une approche moderne basée sur les classes utilitaires.

Frameworks et bibliothèques

– **Next.js 14 (App Router)** : framework principal utilisé pour le développement de l'application, permettant le rendu côté serveur (SSR) et la génération de pages dynamiques.
– **Prisma** : ORM utilisé pour la gestion et l'interaction avec la base de données PostgreSQL.
– **Prisma PostgreSQL Driver** : driver natif optimisé pour la communication avec PostgreSQL.
– **Tailwind CSS** : framework CSS utilitaire permettant un développement rapide et cohérent de l'interface.
– **NextAuth / JWT** : utilisés pour la gestion de l'authentification et des sessions utilisateurs.
– **Chart.js** : bibliothèque utilisée pour la visualisation des données sous forme de graphiques interactifs.
– **OpenAI API** : utilisée pour intégrer des fonctionnalités d'intelligence artificielle au système.
– **Stripe** : solution de paiement en ligne sécurisée intégrée pour la gestion des transactions.
– **Nodemailer** : utilisé pour l'envoi d'emails via protocole SMTP.

Base de données

– PostgreSQL : système de gestion de base de données relationnelle utilisé pour le stockage structuré des données.
– Prisma Data Platform : service d'hébergement de base de données PostgreSQL, facilitant la gestion, la scalabilité et le déploiement.

Outils de développement

– **Visual Studio Code** : éditeur de code principal utilisé pour le développement.
– **Git / GitHub** : utilisés pour la gestion de versions et la collaboration.
– **Vercel** : plateforme utilisée pour le déploiement et l'hébergement de l'application.
– **StarUML** : outil utilisé pour la modélisation UML (diagrammes de cas d'utilisation, classes, séquences).
– **Draw.io** : outil utilisé pour la réalisattion des diagrammes UML aussi.

b) Environnement technique simulé

Le projet est développé dans un environnement local simulant un déploiement réel. Le serveur Flask est exécuté localement et communique avec une base de données PostgreSQL installée en local. Le frontend est directement intégré au backend.

Un script Python d'initialisation permet de générer un jeu de données de test représentatif, incluant des utilisateurs, des produits et des interactions, afin de valider le bon fonctionnement du système.

c) Normes, protocoles et services utilisés

– HTTP/REST : utilisé pour la communication entre le frontend et le backend.
– JSON : format d'échange de données entre le client et le serveur.
– SQL : langage utilisé pour interagir avec la base de données PostgreSQL.
– bcrypt : utilisé pour le hachage sécurisé des mots de passe.
– PEP 8 : convention de codage respectée pour le développement en Python.

5. Planification du projet

a) Découpage du projet

Le projet a été découpé en six phases distinctes, chacune correspondant à un ensemble cohérent de tâches et débouchant sur un livrable identifiable. Ce découpage suit la logique du cycle de développement logiciel classique, adapté aux contraintes d'un projet académique individuel.

Phase 1 : Initialisation : définition des objectifs, étude de faisabilité, choix des technologies (Next.js, PostgreSQL, Prisma) et mise en place de l'environnement de développement.

Phase 2 : Analyse et conception : collecte des besoins, élaboration du cahier des charges, modélisation UML (via StarUML) et conception de l'architecture de l'application.

Phase 3 : Développement backend : mise en place de la base de données PostgreSQL, modélisation avec Prisma, développement des API et intégration des fonctionnalités principales (authentification, logique métier).

Phase 4 : Développement frontend: développement de l'interface utilisateur avec Next.js et Tailwind CSS, et intégration avec le backend.

Phase 5 : Tests et validation : vérification du bon fonctionnement global du système, correction des anomalies et validation des fonctionnalités.

Phase 6 : Déploiement et finalisation : déploiement de l'application sur Vercel, finalisation du projet, rédaction du rapport et préparation de la soutenance.

b) Organisation et suivi

Le suivi du projet est assuré à l'aide d'un tableau de gestion des tâches permettant de visualiser l'avancement des différentes activités. L'utilisation de Git et GitHub permet de gérer les versions du code, de suivre les modifications et d'assurer une organisation structurée du développement. Des points réguliers avec l'encadreur permettent de valider les choix techniques et d'ajuster l'avancement du projet si nécessaire.

c) Objectif de la planification

L'objectif de cette planification est de garantir que l'ensemble des livrables attendus soient produits dans les délais impartis, avec un niveau de qualité satisfaisant. Elle permet également d'identifier et d'anticiper les risques de retard, notamment lors des phases de développement les plus complexes comme l'implémentation des algorithmes de recommandation.

La planification adoptée privilégie la flexibilité : en cas de dépassement sur une tâche, des ajustements peuvent être effectués en réduisant le périmètre des fonctionnalités secondaires plutôt qu'en compromettant la qualité des fonctionnalités essentielles. Cette approche garantit qu'un système fonctionnel et documenté sera livré dans les délais, même si certaines fonctionnalités avancées devaient être reportées.

6. Livrables attendus

a) Livrables techniques

– Code source complet de l'application, organisé localement, incluant le backend basé sur Next.js / Node.js (TypeScript), le frontend (HTML5, CSS / Tailwind, TypeScript / React) ainsi que les scripts d'initialisation et de gestion de la base de données via Prisma ORM.

– Base de données PostgreSQL initialisée avec un jeu de données de test représentatif (utilisateurs, produits, interactions).

– Moteur de recommandation fonctionnel implémentant les trois approches algorithmiques (filtrage collaboratif, filtrage basé sur le contenu, approche hybride), développé en TypeScript.

- Interface utilisateur complète et responsive, permettant la navigation dans le catalogue, la consultation des recommandations et la simulation du processus de commande.
- Interface d'administration permettant la gestion du catalogue et la supervision des utilisateurs.

b) *Livrables de gestion de projet*

- Cahier des charges (le présent document) : formalisation complète des exigences, contraintes, outils et planification du projet.
- Diagrammes UML : diagramme de classes, diagramme d'objets, diagramme de cas d'utilisation, diagrammes de séquence, d'activités, d'état-transition et de collaboration.
- WBS (Work Breakdown Structure) : décomposition hiérarchique de l'ensemble des tâches du projet.
- Diagramme de Gantt : représentation graphique du planning du projet avec les dépendances entre tâches.
- Matrice de gestion des risques : identification, évaluation et plan de mitigation des principaux risques du projet.
- Schémas d’architecture : diagrammes réalisés illustrant l’architecture globale du système.

c) *Livrables finaux*

- Rapport de projet complet : document détaillant le contexte, les objectifs, les choix techniques, la conception, la réalisation et les résultats obtenus.
- Présentation de soutenance : support de présentation synthétisant les points clés du projet pour la soutenance orale.
- Démonstration fonctionnelle : application déployée et fonctionnelle, présentée lors de la soutenance pour illustrer les fonctionnalités développées.
- Fichier README : guide d'installation et de lancement de l'application, permettant à toute personne de reproduire l'environnement de développement.

d) *Objectif global des livrables*

L'ensemble de ces livrables vise à démontrer la maîtrise du cycle complet de développement d'un projet informatique, depuis l'analyse des besoins jusqu'au déploiement d'une solution fonctionnelle. Chaque livrable répond à un objectif précis : les livrables techniques prouvent la capacité à implémenter des solutions algorithmiques et logicielles complexes, les livrables de gestion de projet témoignent de la rigueur méthodologique, et les livrables finaux permettent de communiquer et de valoriser le travail réalisé.

La qualité de l'ensemble de ces livrables est conditionnée par le respect du cahier des charges définies dans ce document, qui sert de référence tout au long du projet pour orienter les décisions et évaluer la conformité du travail produit aux objectifs initialement fixés.

II. Planification du projet

La planification du projet constitue une étape essentielle dans la réussite du système de recommandation. Elle permet d'organiser les différentes activités, d'estimer les ressources nécessaires et d'anticiper les contraintes et les risques. Une planification rigoureuse garantit le respect des délais et la qualité du produit final.

A. Méthodologie Scrum

1. Contexte du projet

Dans le cadre du module de Gestion de Projet Logiciel, le présent rapport documente la mise en oeuvre de la méthode Scrum pour le développement d'un **Système de Recommandation de Produits**. Ce système a pour vocation de proposer à chaque utilisateur des suggestions de produits personnalisées, générées à partir de son historique d'achats, de ses préférences déclarées et de son comportement de navigation.

La méthode Scrum a été retenue en raison de la nature itérative inhérente à ce type de projet : un moteur de recommandation requiert des cycles d'entraînement et d'évaluation répétés, et les retours des utilisateurs constituent un levier essentiel d'amélioration du modèle. Le cadre Scrum permet ainsi de livrer des fonctionnalités opérationnelles à l'issue de chaque sprint, tout en conservant la capacité d'adapter les priorités de développement en fonction des résultats observés.

L'équipe projet est composée de quatre membres ayant des rôles différents afin d'assurer le bon déroulement du projet.

2. Formation de l'équipe Scrum

L'équipe a été constituée en respectant les trois rôles fondamentaux du cadre Scrum. La répartition des responsabilités a été établie dès la phase de lancement du projet, afin de garantir une organisation claire et une prise de décision efficace tout au long du développement.

Rôle	Membre	Responsabilités
Product Owner	Dr. Moubarek	Définit les besoins et priorise les fonctionnalités du projet.
Scrum Master	Abdinasser	Organise le travail, facilite l'équipe et veille au respect de Scrum.
Equipe de developpement	Tous les membres de l'équipe	Réalise le projet (développement, tests, conception).

3. Création du Product Backlog

Priorité	Fonctionnalité
1	Créer et gérer les comptes utilisateurs (inscription, connexion)
2	Collecter et stocker les données utilisateur (profil, préférences)
3	Enregistrer les interactions utilisateur (clics, vues, achats)
4	Afficher le catalogue des produits
5	Mettre en place le moteur de recommandation (algorithme de base)
6	Générer des recommandations personnalisées
7	Optimiser les recommandations (hybride : contenu + collaboratif)
8	Mettre en place un système de cache pour améliorer les performances
9	Afficher les recommandations sur la page d'accueil
10	Tester et améliorer le système (feedback utilisateur)

4. Sprint Planning (Planification du Sprint)

Au début de chaque Sprint, l'équipe organise une réunion appelée **Sprint Planning**.

Pendant cette réunion :

● Le Product Owner présente les fonctionnalités prioritaires du système (recommandation, utilisateurs, interface, etc.)

● L'équipe sélectionne les tâches réalisables durant le Sprint

● Les tâches sont découpées en activités plus petites pour faciliter leur réalisation

Exemple de Sprint Planning:

Sprint 1 (Phase de conception)

● Analyse des besoins du système

● Réalisation des diagrammes UML :

- diagramme de classes
- diagramme de cas d'utilisation
- diagramme de séquence
- diagramme d'activités
- diagramme d'état-transition
- diagramme d'objets
- diagramme de collaboration

● Conception du prototype de l'interface utilisateur

Sprint 2 (Phase de développement)

● Création de la base de données

● Développement du frontend (interface utilisateur)

● Développement du backend (API, logique métier)

● Implémentation du moteur de recommandation

● Intégration des fonctionnalités :

- authentification
- affichage des produits
- recommandations personnalisées

Schéma Sprint Planning:

Product Backlog
│ sélection des tâches
▼
Sprint Backlog
│
▼
Travail de l'équipe
pendant le Sprint

Dans notre projet, la planification des sprints a permis d'organiser efficacement les tâches entre la phase de conception et la phase de développement, en assurant une progression claire et structurée du système de recommandation.

5. Développement du Sprint

Les activités conduites au cours de chaque sprint ont inclus :

- Le développement du code back-end (moteur de recommandation, APIs REST)
- La conception et l'intégration de l'interface utilisateur
- L'entraînement et l'évaluation des modèles de recommandation
- Les tests fonctionnels et la correction des anomalies identifiées

Une communication constante a été maintenue entre les membres de l'équipe afin d'anticiper les dépendances techniques entre les différents modules du système et d'assurer la cohérence des livrables produits à l'issue de chaque sprint.

6. Daily Scrum

Un Daily Scrum d'une durée strictement limitée à quinze minutes a été tenu chaque jour ouvrable, en début de matinée. Cette réunion courte a permis de synchroniser l'ensemble de l'équipe sur l'état d'avancement des tâches et de détecter rapidement les obstacles susceptibles de compromettre l'objectif du sprint.

Chaque membre a répondu successivement aux trois questions suivantes :

1. Qu'a-t-il été accompli depuis le dernier Daily Scrum ?
2. Quelles tâches sont prévues d'ici le prochain Daily Scrum ?
3. Des obstacles freinent-ils l'avancement ?

Les obstacles signalés ont été pris en charge par le Scrum Master en dehors de la réunion, afin de préserver la concentration de l'équipe sur les tâches en cours. Cette discipline quotidienne a permis de maintenir une progression cohérente sur l'ensemble des tâches du sprint et de limiter les risques de dépassement.

7. Sprint Review

À l'issue de chaque sprint, une Sprint Review d'environ deux heures a été organisée en présence du Product Owner et des utilisateurs représentatifs. L'objectif de cette réunion était de présenter l'incrément livrable produit durant le sprint et de recueillir les retours des parties prenantes.

À titre d'exemple, lors de la Sprint Review du Sprint 1, les éléments suivants ont été présentés :

- Le moteur de recommandation initial avec filtrage par catégorie de produits
- L'interface de navigation avec affichage des suggestions personnalisées
- Les résultats des premiers tests de validation du modèle

Les retours recueillis lors de cette revue ont conduit à affiner la priorisation du Product Backlog pour le Sprint 2, notamment en renforçant les fonctionnalités de personnalisation en temps réel, jugées prioritaires par les utilisateurs consultés.

8. Sprint Retrospective — Rétrospective

Après chaque Sprint Review, une rétrospective interne d'une durée d'une heure trente a été conduite par le Scrum Master. Cette réunion avait pour objectif d'analyser le déroulement du sprint écoulé et d'identifier des axes d'amélioration concrets à mettre en oeuvre dès le sprint suivant.

Les échanges ont porté sur trois axes structurants :

- Les pratiques ayant contribué positivement à l'atteinte de l'objectif du sprint
- Les dysfonctionnements ou difficultés rencontrés au cours du sprint
- Les actions correctives à déployer lors du sprint suivant

À l'issue de la rétrospective du Sprint 1, il a été décidé de renforcer la couverture de tests avant toute mise en production de fonctionnalité, et d'améliorer la communication relative aux dépendances entre tâches, afin de prévenir les blocages constatés en fin de sprint.

9. Livraison progressive du système

Conformément aux principes du cadre Scrum, le système de recommandation a été développé et livré de manière progressive. Chaque sprint a permis de produire un incrément fonctionnel validé par l'équipe et démontré au Product Owner.

Étant donné la durée limitée du projet, les livraisons ont été organisées en deux phases principales : une phase de conception et une phase de développement. Chaque phase a abouti à un livrable concret permettant de valider l'avancement du projet.

Sprint	Fonctionnalité livrée
Sprint 1	Modélisation complète du système (diagrammes UML : classe, cas d'utilisation, séquence, activité, état-transition, objet, collaboration) + prototype de l'interface utilisateur
Sprint 2	Application web complète (frontend + backend) avec moteur de recommandation fonctionnel, gestion des utilisateurs, affichage des produits et recommandations personnalisées

B. Estimations des tâches

L'estimation des tâches constitue une étape fondamentale dans la planification du projet, car elle permet d'identifier de manière précise les différentes activités à réaliser ainsi que le temps nécessaire à leur exécution. Cette démarche vise à assurer une organisation efficace du travail et à garantir une répartition cohérente des efforts tout au long du projet.

Dans le cadre de ce projet de développement d'un système de recommandation, les tâches ont été identifiées en fonction des différentes phases du cycle de vie du projet.

L'estimation de ces tâches a été réalisée en tenant compte de plusieurs critères, notamment la complexité technique, le niveau de maîtrise des outils utilisés, ainsi que les contraintes temporelles du projet. Une approche empirique a été adoptée afin d'obtenir des estimations réalistes.

Tableau d'estimations des tâches

Phase	Tâche	Durée
Analyse et planification	Recueil des besoins et définition du périmètre	3 jours
	Rédaction du cahier des charges	2 jours
	Choix des technologies et outils	1 jour
Conception du système	Conception de l'architecture globale	3 jours
	Modélisation UML	2 jours
	Conception de l'interface utilisateur (maquette)	2 jours
Développement	Développement de l'algorithme de recommandation	5 jours
	Développement du backend Flask	4 jours
	Développement du site web (HTML/CSS/JS)	4 jours
Tests, validation et rédaction du rapport	Tests unitaires et d'intégration	3 jours
	Tests système et validation	2 jours
	Finalisation et rendu du rapport	2 jours
Finalisation et mise en ligne	Corrections finales et mise en ligne du site	1 jour
	Préparation de la soutenance	1 jour

C. Découpage WBS

Pour structurer le projet et organiser clairement les différentes tâches à réaliser, un WBS (Work Breakdown Structure) a été élaboré. Ce découpage permet de visualiser l'ensemble des activités, de suivre l'avancement du projet et de faciliter la planification des ressources et des responsabilités pour chaque phase.

Le schéma ci-dessous présente la structure hiérarchique des tâches, allant du projet global jusqu'aux sous-tâches détaillées pour chaque phase : analyse, conception, implémentation, tests, et déploiement/maintenance.

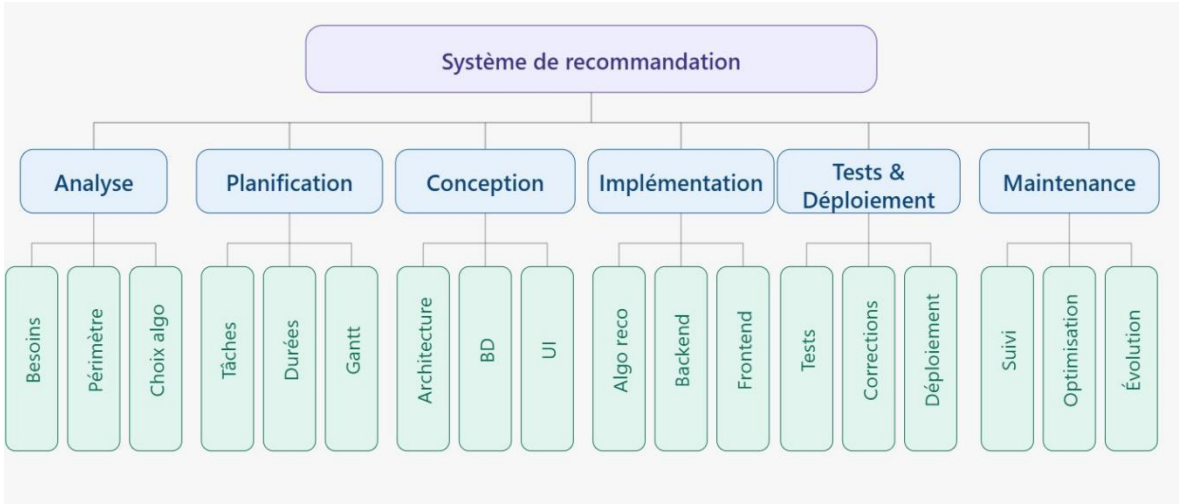
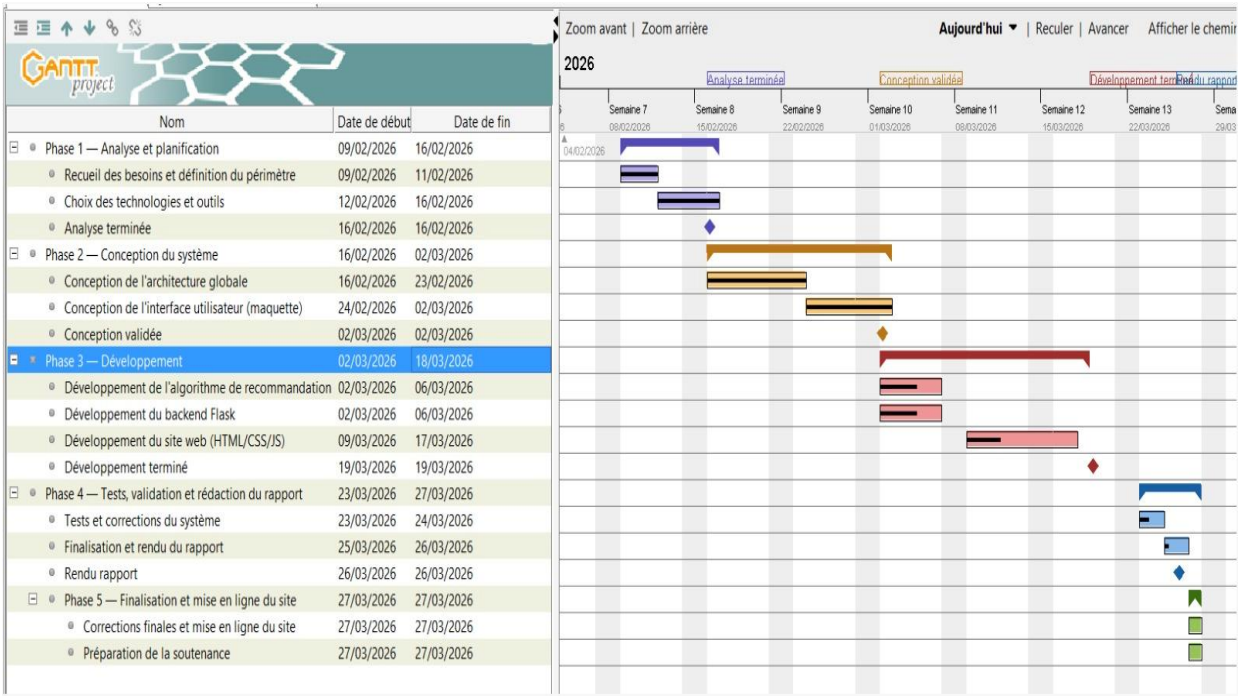


Figure 1 : *WBS : Structure du Projet de Développement du Système de Recommandation de Produits*

D. Diagramme de Gantt

Le diagramme de Gantt présenté ci-dessous offre une représentation visuelle de la planification temporelle du projet. Il permet de suivre l'enchaînement des différentes tâches, de visualiser leur durée ainsi que leurs interdépendances, facilitant ainsi la gestion et le suivi de l'avancement global.



Nom	Date de début	Date de fin
Phase 1 — Analyse et planification	09/02/2026	16/02/2026
Recueil des besoins et définition du périmètre	09/02/2026	11/02/2026
Choix des technologies et outils	12/02/2026	16/02/2026
Analyse terminée	16/02/2026	16/02/2026
Phase 2 — Conception du système	16/02/2026	02/03/2026
Conception de l'architecture globale	16/02/2026	23/02/2026
Conception de l'interface utilisateur (maquette)	24/02/2026	02/03/2026
Conception validée	02/03/2026	02/03/2026
Phase 3 — Développement	02/03/2026	18/03/2026
Développement de l'algorithme de recommandation	02/03/2026	06/03/2026
Développement du backend Flask	02/03/2026	06/03/2026
Développement du site web (HTML/CSS/JS)	09/03/2026	17/03/2026
Développement terminé	19/03/2026	19/03/2026
Phase 4 — Tests, validation et rédaction du rapport	23/03/2026	27/03/2026
Tests et corrections du système	23/03/2026	24/03/2026
Finalisation et rendu du rapport	25/03/2026	26/03/2026
Rendu rapport	26/03/2026	26/03/2026
Phase 5 — Finalisation et mise en ligne du site	27/03/2026	27/03/2026
Corrections finales et mise en ligne du site	27/03/2026	27/03/2026
Préparation de la soutenance	27/03/2026	27/03/2026

Figure 2 : *Diagramme de Gantt du projet montrant la planification des tâches et la chronologie des différentes phases.*

E. Gestion des risques

La gestion des risques est un élément clé du projet, permettant d'anticiper les problèmes susceptibles d'impacter le respect des délais, la qualité du système ou la réussite des différentes phases. Les risques identifiés sont regroupés en trois catégories : techniques, organisationnels et humains, avec des mesures concrètes prévues pour limiter leur impact.

Type de risque	**Description**	**Probabilté**	**Impact**	**Mesures d'atténuation**
Technique	Difficulté d'intégration entre Next.js, Prima et PostgreSQL	Moyenne	Elevé	Test progressifs, documentation, optimisation des requêtes
Technique	Risque liés aux API externes	Moyenne	Moyen	Gestion des erreurs, surveillance des quotas, solutions alternatives prévus
Technique	Performances des pages dynamiques avec gros volume de données	Faible	Moyen	Optimisation du code, mise en cache
Organisationnel	Retards dans certaines phrases du développement	Moyenne	Elevé	Planning réaliste, suivi régulier, priorisation des taches critique
Organisationnel	Dépendances entre phases non respectées	Faible	Moyen	Planification claire
Humain	Manque d'expérience sur certaines technologies	Faible	Moyen	Documentation technique
Humain	Surcharge de travail ou baisse de motivation	Faible	Moyen	Calendrier réaliste, pauses et organisations personnelle

III. Modélisation du système

A. Architecture

Le système a été conçu en tenant compte des besoins fonctionnels et non fonctionnels identifiés. Plusieurs styles architecturaux ont été étudiés afin de choisir celui qui correspond le mieux aux exigences du projet, notamment en termes de performance, de maintenabilité et d'évolutivité.

Parmi les architectures analysées figurent l'architecture en couches, l'architecture client-serveur et l'architecture microservices. Après comparaison, l'architecture en couches a été retenue, car elle offre une séparation claire des responsabilités et s'adapte parfaitement à une application web moderne basée sur Next.js.

1. Architecture en couches

Le système est structuré en différentes couches fonctionnelles : présentation, logique métier et accès aux données. Cette organisation permet d'isoler les composants, de faciliter la maintenance et de rendre le développement plus structuré. Les modifications dans une couche n'impactent pas les autres couches, ce qui simplifie les évolutions futures.

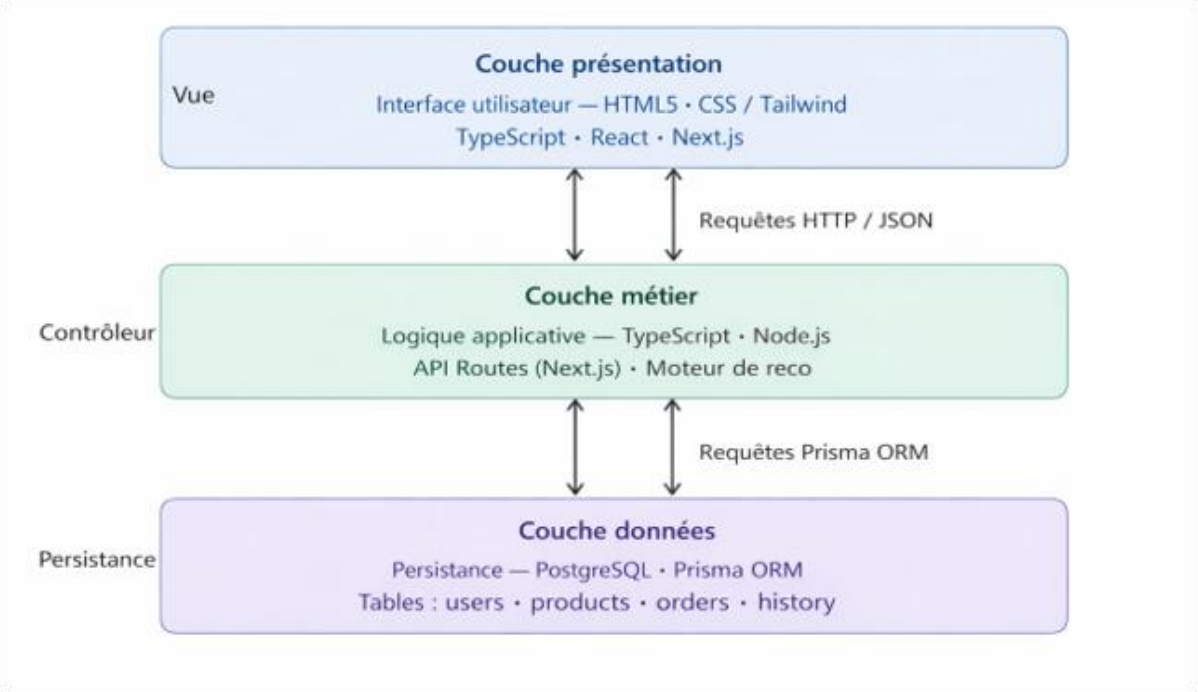


Figure 3 : Architecture en couche du système Nexora

2. Architecture client-serveur

Le modèle client-serveur assure la communication entre le client et le serveur. Le client envoie les requêtes, le serveur les traite et renvoie les réponses. Ce modèle centralise la gestion des données et facilite le contrôle des accès, tout en garantissant une communication standardisée entre les composants.

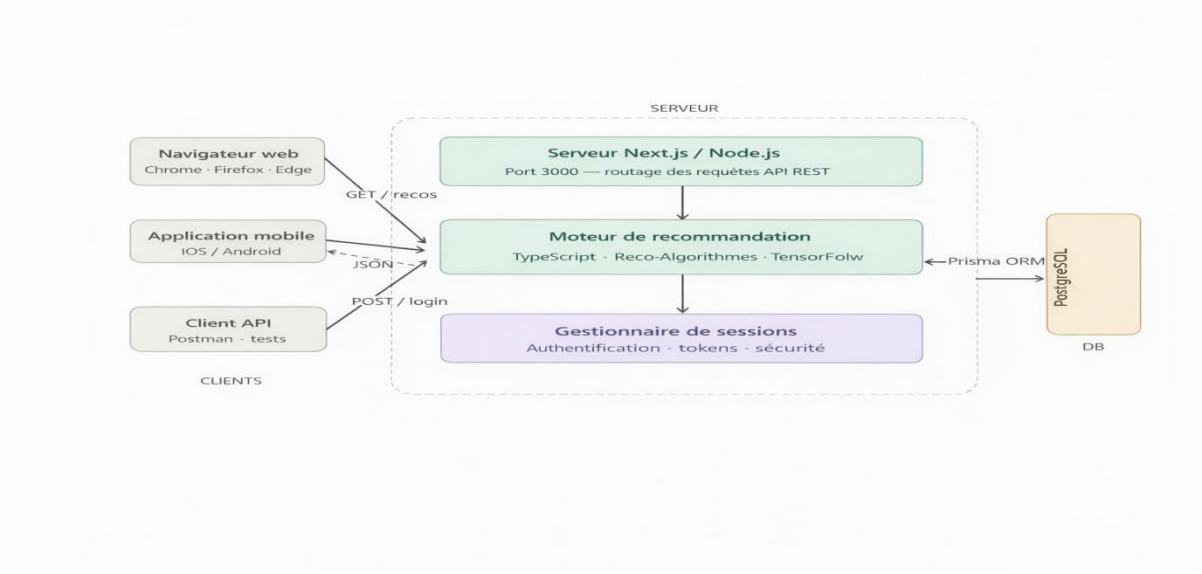


Figure 4 : Architecture client-serveur

3. Architecture micro-service

Certaines fonctionnalités critiques ont été envisagées sous forme de micro-services, chaque service gérant une fonctionnalité spécifique du système. Cette approche permet de découpler les modules, de faciliter les mises à jour indépendantes et d'améliorer la scalabilité du système.

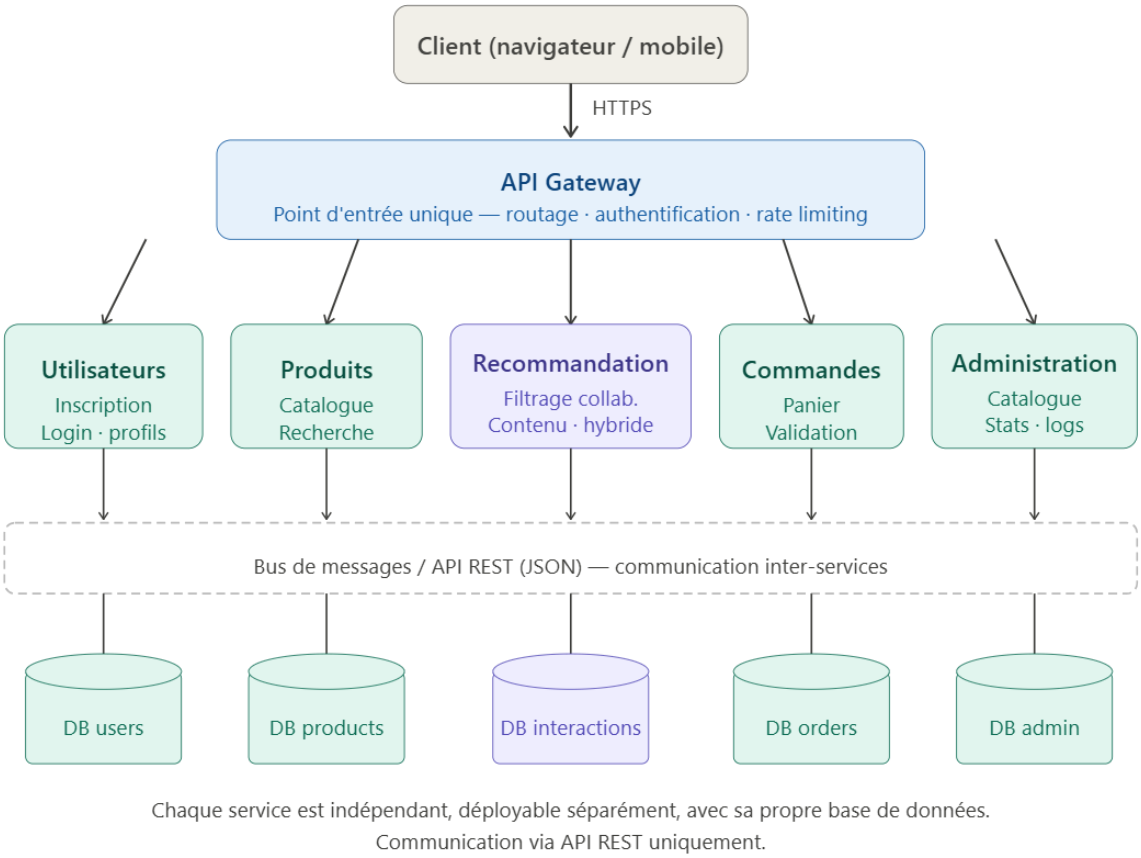


Figure 5 : Architecture micro-service

B. Conception UML

La conception UML du système fournit une représentation structurée de sa configuration et de son fonctionnement. Elle permet de visualiser les composants du système, d'assurer une compréhension commune au sein de l'équipe et de soutenir la planification du développement. Les diagrammes sont organisés en **diagrammes statiques**, décrivant la structure du système, et **diagrammes dynamiques**, illustrant son comportement et le déroulement des interactions.

1. Diagrammes statiques

Les diagrammes statiques représentent la structure du système et les relations entre ses composants. Ils permettent de visualiser les entités principales, leurs attributs et méthodes, ainsi que les dépendances et associations entre elles. Ces diagrammes constituent une base solide pour l'implémentation et la maintenance du système, tout en facilitant la communication entre les membres de l'équipe.

a) Diagramme de classe

Le diagramme de classes ci-dessous représente la structure statique du système e-commerce de recommandation. Il modélise l'ensemble des classes métier du système, leurs attributs,

leurs méthodes ainsi que les relations qui les unissent (associations, compositions, héritages). Chaque classe correspond à une entité réelle du système, et les cardinalités indiquées sur les relations précisent le nombre d'instances pouvant être associées entre deux classes. Ce diagramme constitue le fondement de l'architecture logicielle du projet et sert de référence pour l'implémentation de la base de données et du code source.

- En raison de la richesse du modèle et du nombre important de classes, le diagramme de classes a été volontairement divisé en deux parties afin de garantir une meilleure lisibilité.

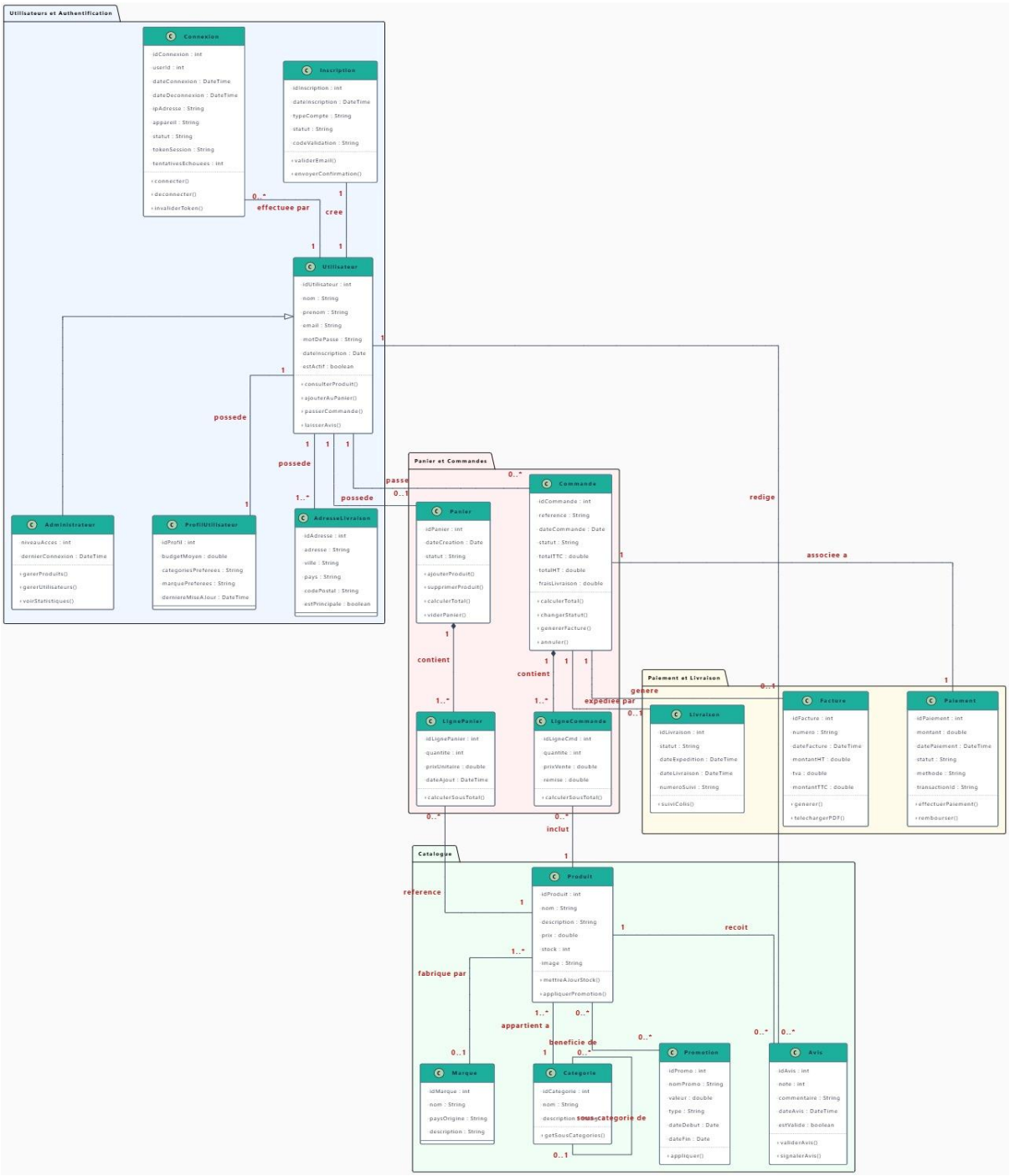


Figure 6 : Diagramme de classes du système.

Cette première partie regroupe les classes fondamentales de l'application. On y trouve la gestion des utilisateurs et de l'authentification (inscription, connexion, profil), le catalogue produits (produits, catégories, marques, promotions, avis), le cycle d'achat (panier, lignes de panier, commandes) ainsi que le traitement des paiements et de la livraison. Ces classes forment le socle transactionnel du système sur lequel repose l'ensemble des fonctionnalités.

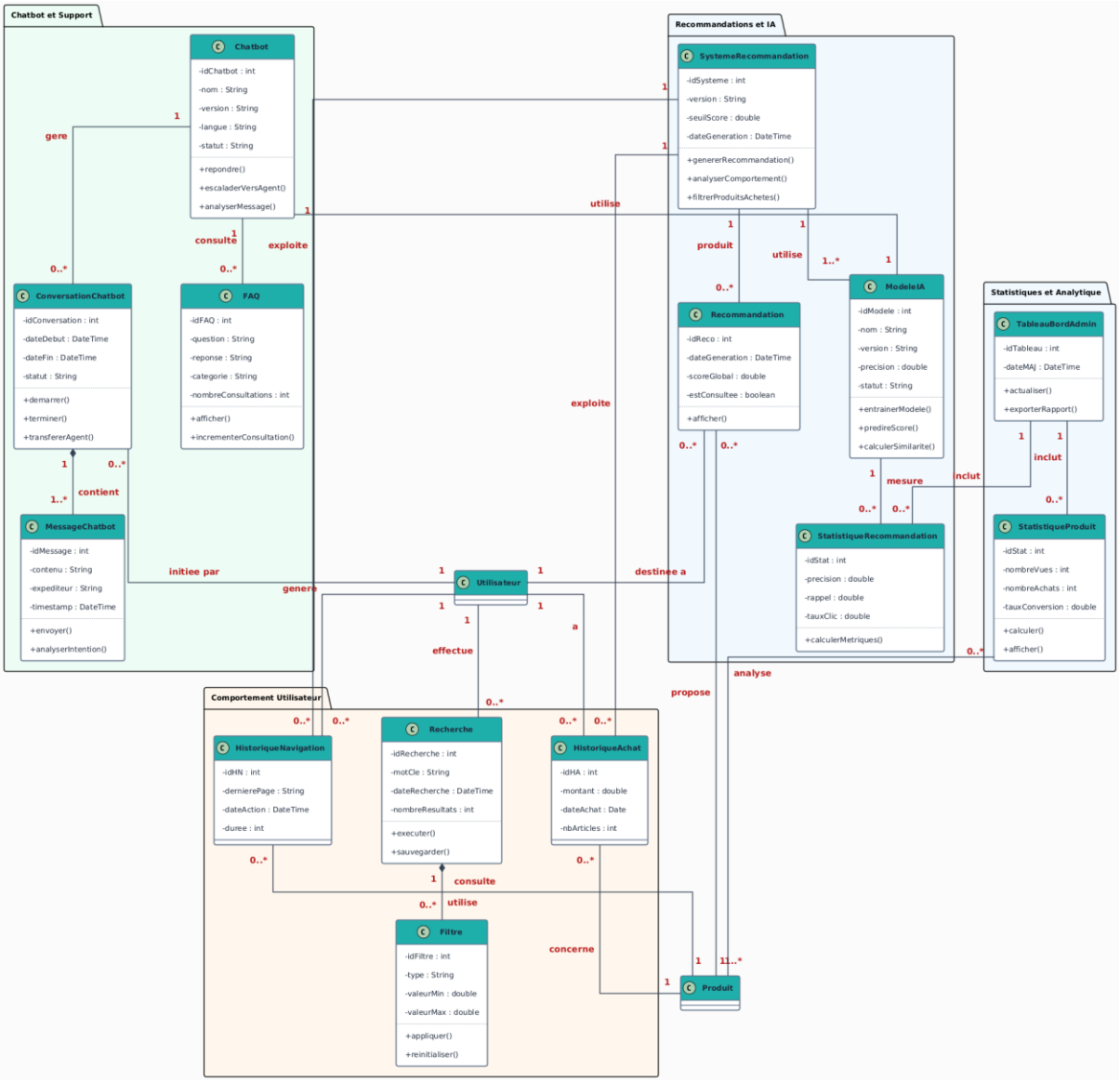


Figure 7 : Diagramme de classes du système.

Cette seconde partie présente les classes liées à l'intelligence du système. Le package Comportement Utilisateur capture les actions de navigation et d'achat qui alimentent le moteur de recommandation. Le package Recommandations & IA contient les classes responsables de la génération et de l'évaluation des recommandations personnalisées. Le package Chatbot & Support modélise les interactions conversationnelles avec l'utilisateur. Enfin, le package Statistiques & Analytique permet à l'administrateur de superviser les performances globales du système.

b) *Diagramme d'objet*

Le diagramme d'objet est une instance du diagramme de classes. Il représente un instantané du système à un moment précis, en montrant des objets concrets avec leurs valeurs réelles plutôt que des classes abstraites. Contrairement au diagramme de classes, les relations entre objets ne portent ni cardinalités ni labels, car elles illustrent des liens déjà établis entre des instances spécifiques. Le scénario choisi suit le parcours complet d'un utilisateur réel, Ahmed Diallo, depuis sa connexion jusqu'à la réception de recommandations personnalisées.

- De même, le diagramme d'objet a été scindé en deux parties pour une meilleure visibilité.

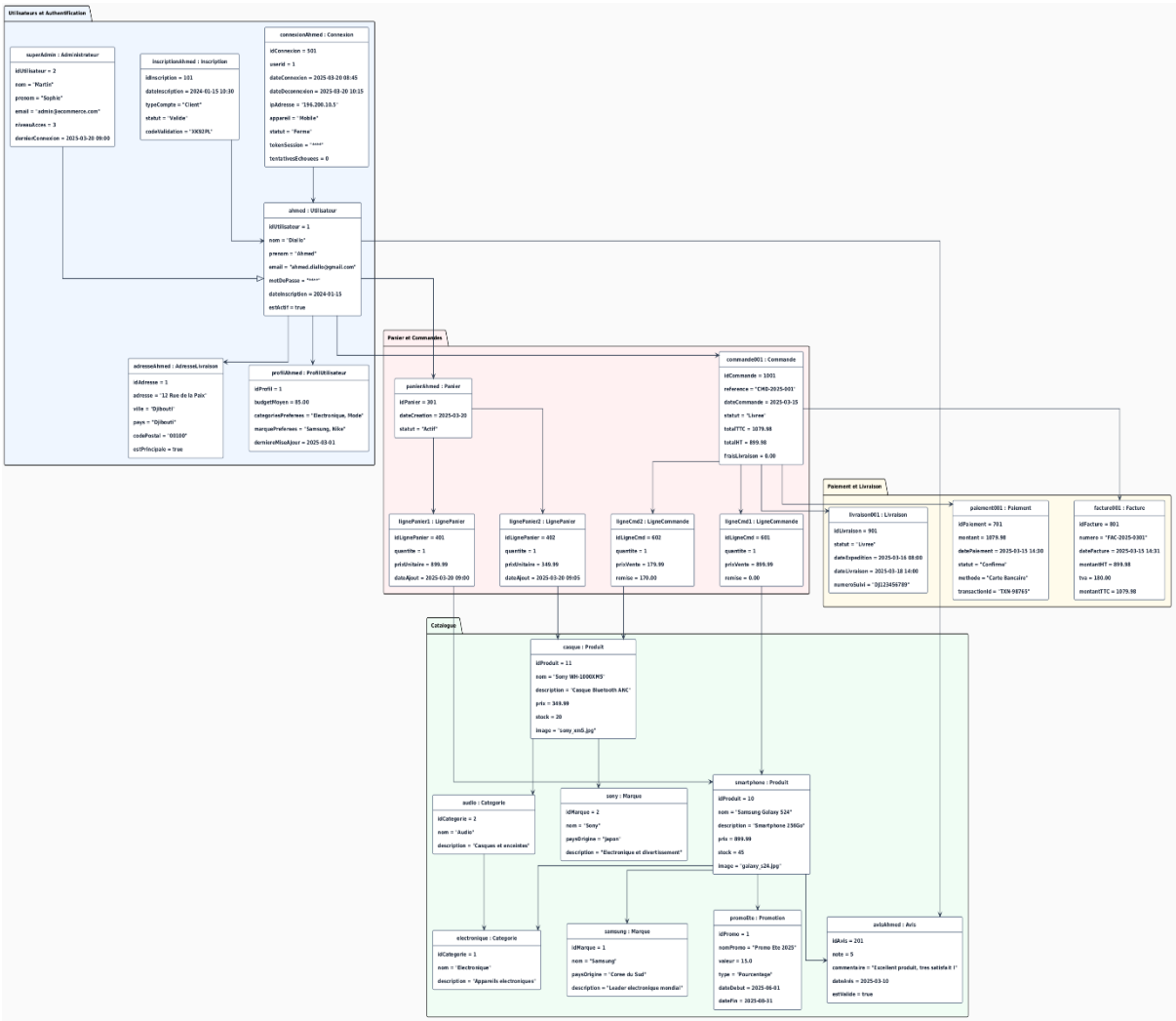


Figure 8 : Diagramme d'objets du système.

Cette première partie instancie les classes du socle transactionnel. On y observe l'objet ahmed de type Utilisateur avec ses informations réelles, son inscription, sa connexion, son profil et son adresse. On retrouve également deux produits concrets (Samsung Galaxy S24 et Sony WH-1000XM5), leurs catégories et marques associées, ainsi que le panier actif d'Ahmed, sa

commande finalisée, le paiement effectué par carte bancaire, la facture générée et la livraison correspondante.

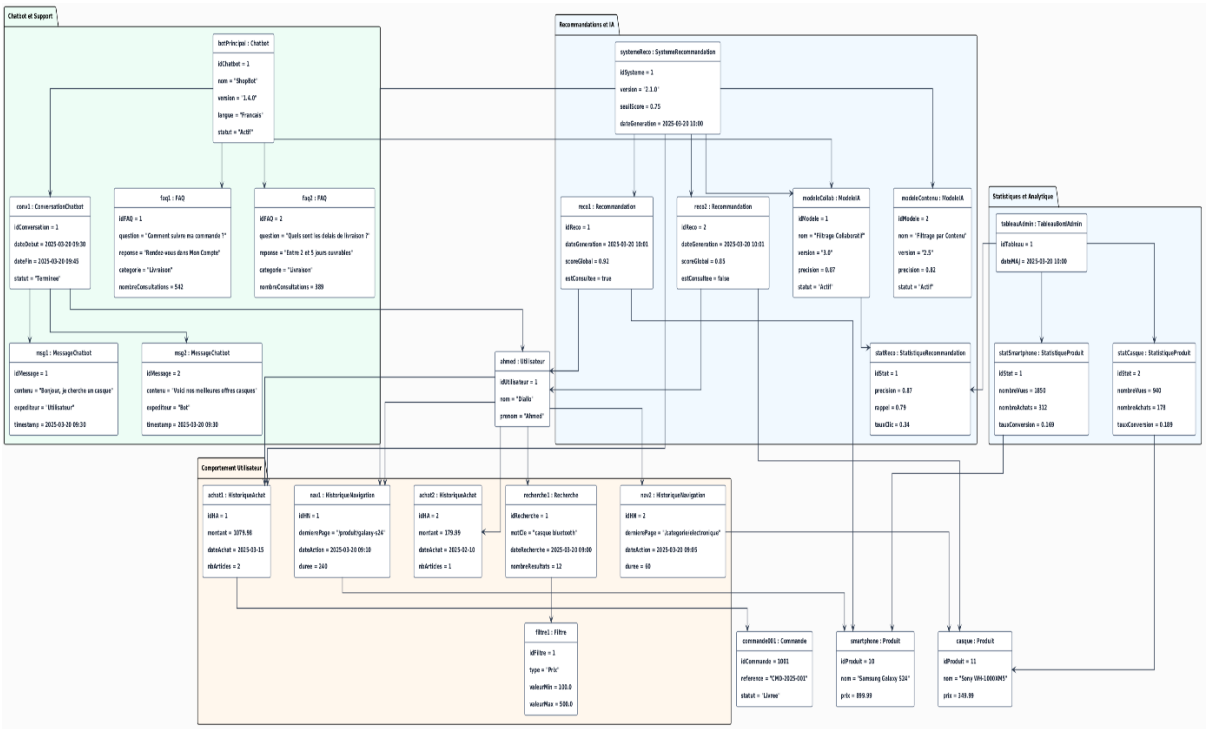


Figure 9 : Diagramme d'objets du système.

Cette seconde partie instancie les classes intelligentes du système. On y voit les historiques de navigation et d'achat d'Ahmed qui alimentent le SystemeRecommandation. Deux modèles d'IA sont actifs : le filtrage collaboratif et le filtrage par contenu, qui génèrent deux recommandations personnalisées proposées à Ahmed. Le chatbot ShopBot engage une conversation avec l'utilisateur en s'appuyant sur la FAQ et le modèle IA. Enfin, le tableau de bord administrateur agrège les statistiques des produits et des recommandations pour évaluer les performances du système.

c) *Diagramme de cas d'utilisation*

Le diagramme de cas d'utilisation présente les interactions entre les utilisateurs et le système, mettant en évidence les fonctionnalités principales et leur portée. Il sert de guide pour le développement des fonctionnalités et la validation des besoins.

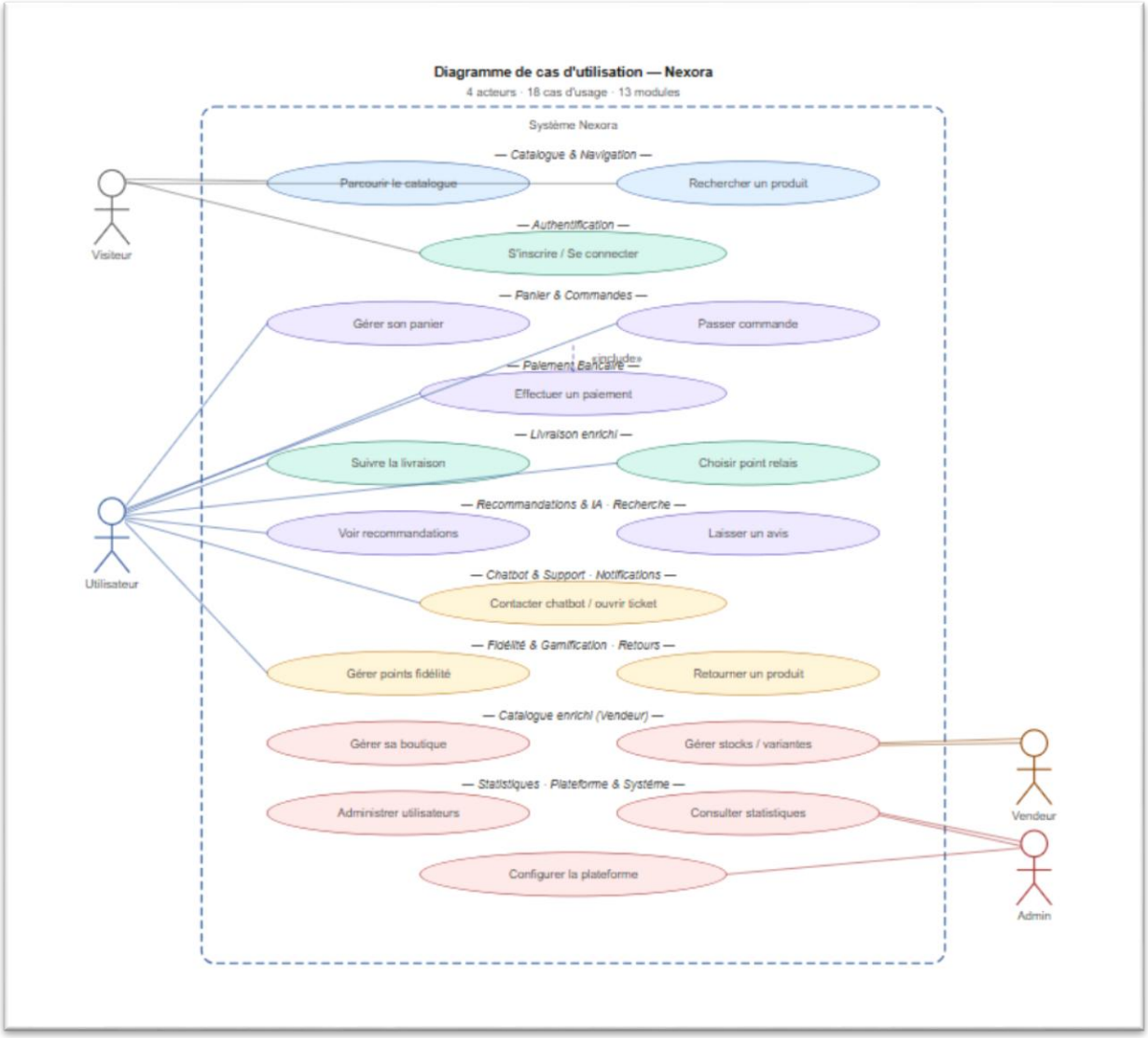


Figure 10 : Diagramme de cas d'utilisation du système.

Ce diagramme illustre les interactions entre les acteurs (visiteur, utilisateur, vendeur, administrateur) et le système Nexora. Il présente les différentes fonctionnalités offertes, comme consulter le catalogue, passer une commande ou gérer les produits. Les cas d'utilisation sont organisés en modules tels que panier, paiement, livraison et recommandations. Il permet de comprendre les besoins fonctionnels du système du point de vue utilisateur. Ce diagramme est essentiel pour définir les fonctionnalités principales à implementer.

2. Diagrammes dynamiques

Les diagrammes dynamiques illustrent le comportement du système et la séquence des actions entre les différents composants. Ils permettent d'anticiper le flux d'exécution des fonctionnalités, de détecter les incohérences et d'optimiser la communication entre objets.

a) Diagramme de séquence

Le diagramme de séquence décrit l'ordre chronologique des interactions entre les objets pour réaliser une fonctionnalité précise. Il facilite la compréhension des échanges et de la coordination entre composants.

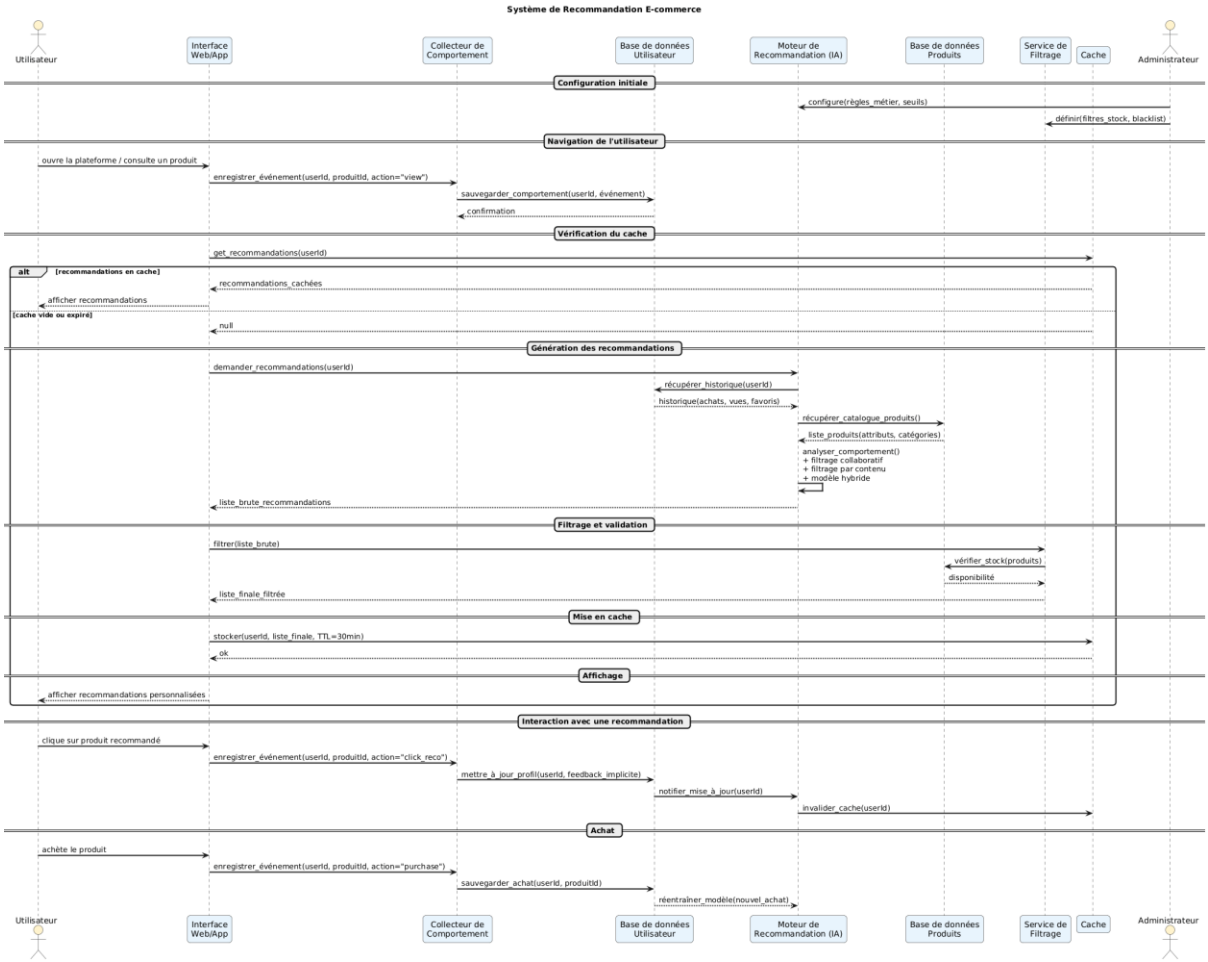


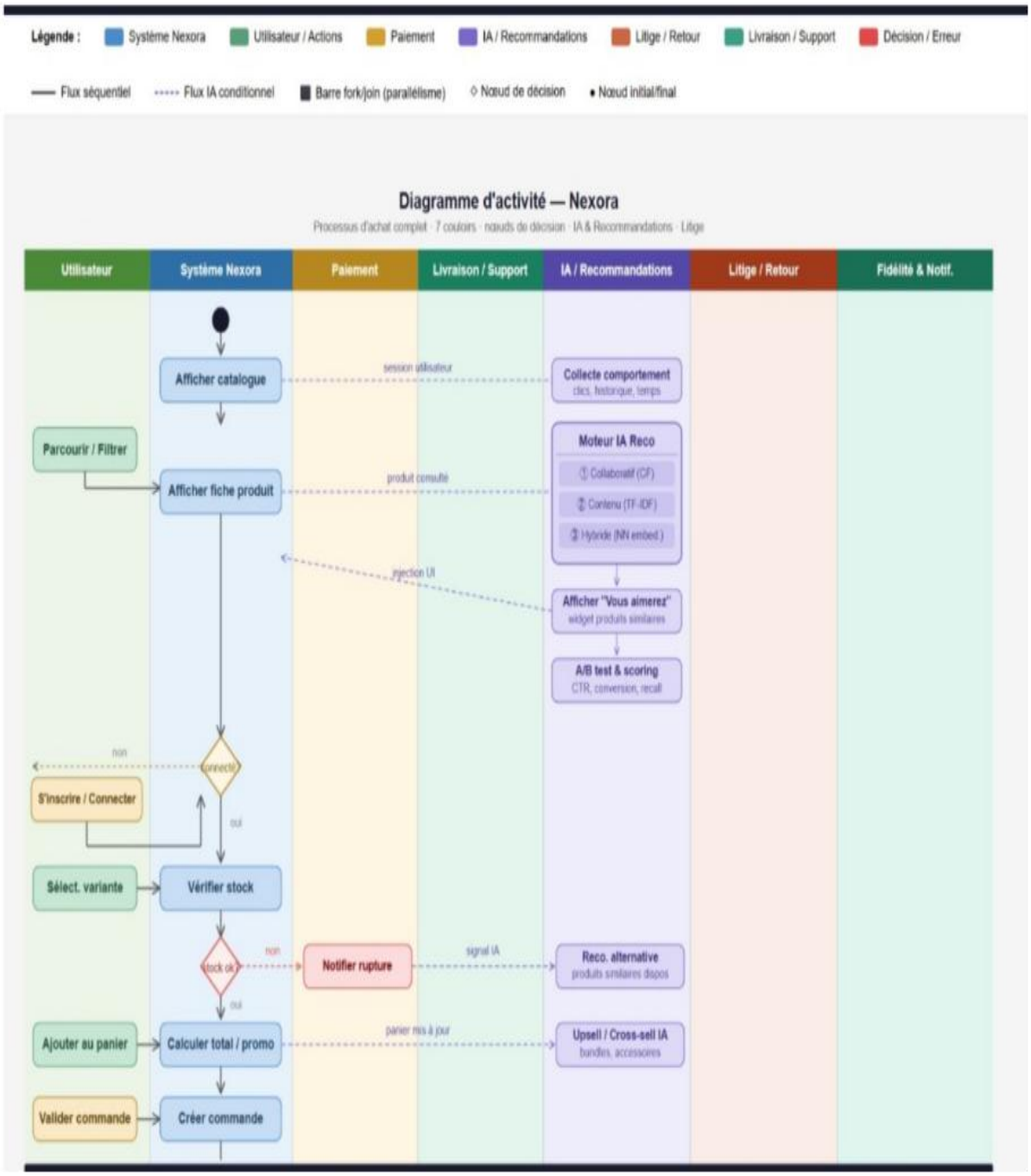
Figure 11 : Diagramme de séquence du module de recommandation.

Ce diagramme de séquence représente le processus d'un système de recommandation de produits dans une plateforme e-commerce. Il illustre les interactions entre l'utilisateur, l'administrateur, et les différents composants du système. Lorsqu'un administrateur configure le système de recommandation sur la plateforme e-commerce, il définit les paramètres, tels que les algorithmes à utiliser et les critères influençant les suggestions de produits. Une fois l'utilisateur connecté, le système collecte des données telles que son historique d'achats, ses préférences de navigation et ses évaluations de produits, qui sont stockées dans une base de données. Ces informations sont ensuite traitées par le calculateur de recommandations, qui applique des algorithmes de filtrage pour proposer des produits personnalisés. Les recommandations sont affichées à l'utilisateur, qui peut choisir d'ajouter des produits à son panier ou de les acheter. À chaque interaction (ajout au panier, achat), le système ajuste ses suggestions en fonction des nouvelles actions de l'utilisateur, affinant ainsi les recommandations de manière continue. Les produits recommandés sont alors constamment mis à jour en fonction des comportements et préférences de l'utilisateur. Grâce à ce processus

dynamique, le système devient de plus en plus précis et personnalisé, offrant une expérience d'achat optimisée et adaptée aux besoins individuels de chaque utilisateur.

b) *Diagramme d'activités*

Le diagramme d'activités illustre le flux des opérations et les décisions prises à chaque étape d'un processus. Il est particulièrement utile pour identifier les étapes critiques et les points de contrôle dans le système.



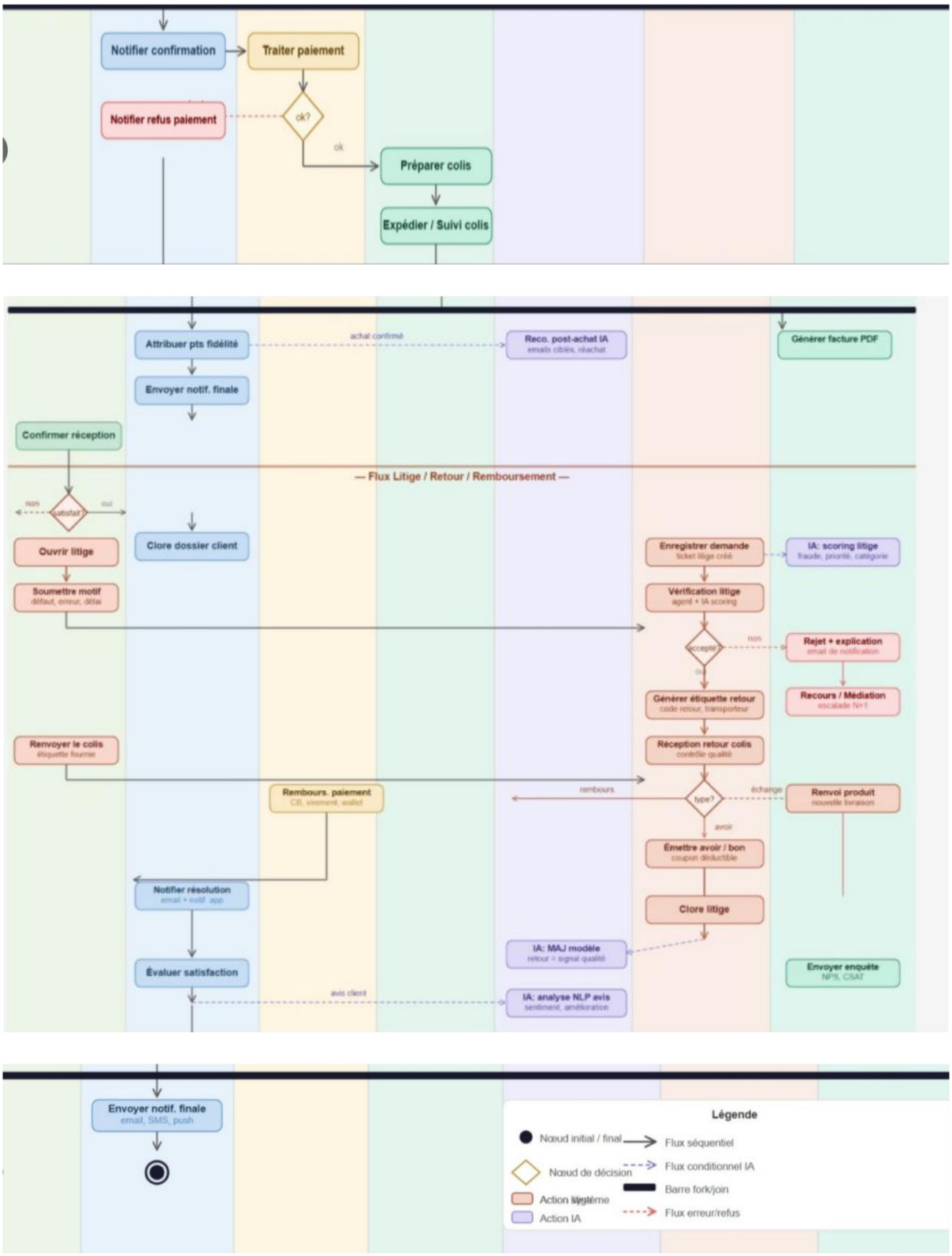


Figure 12 : Diagramme d'activités pour le processus de gestion des recommandations.

Le diagramme d'activité représente le **processus de gestion des recommandations** dans le système e-commerce. Il décrit de manière séquentielle les différentes étapes suivies depuis l'interaction de l'utilisateur jusqu'à l'affichage des produits recommandés.

Le processus débute lorsque **l'utilisateur accède à la plateforme** et effectue une action, comme consulter des produits, naviguer dans le catalogue ou effectuer un achat. Ces actions déclenchent la **collecte des données utilisateur**, notamment l'historique de navigation, les préférences et les interactions précédentes.

Ensuite, ces données sont **stockées dans la base de données**, puis transmises au **moteur de recommandation**. À ce niveau, le système applique différents algorithmes (filtrage collaboratif, basé sur le contenu ou hybride) afin d'analyser les comportements et de déterminer les produits les plus pertinents.

Une étape de **prise de décision** intervient ensuite : le système vérifie si les données disponibles sont suffisantes.

- Si oui, il génère des recommandations personnalisées.
- Sinon, il propose des recommandations par défaut (par exemple basées sur le contenu ou les produits populaires).

Les recommandations générées sont ensuite **affichées à l'utilisateur** sur différentes interfaces (page d'accueil, fiche produit, etc.). L'utilisateur peut alors interagir avec ces suggestions (cliquer, ajouter au panier, acheter).

Enfin, chaque nouvelle interaction est **réinjectée dans le système**, ce qui permet de mettre à jour les données et d'améliorer continuellement la qualité des recommandations. Le processus forme ainsi une **boucle continue d'apprentissage et d'adaptation**.

c) Diagramme d'état-transition

Le diagramme d'état-transition représente les différents états possibles des objets et les événements déclenchant leur passage d'un état à un autre. Il permet de modéliser le comportement dynamique des modules et de prévoir la gestion des exceptions.

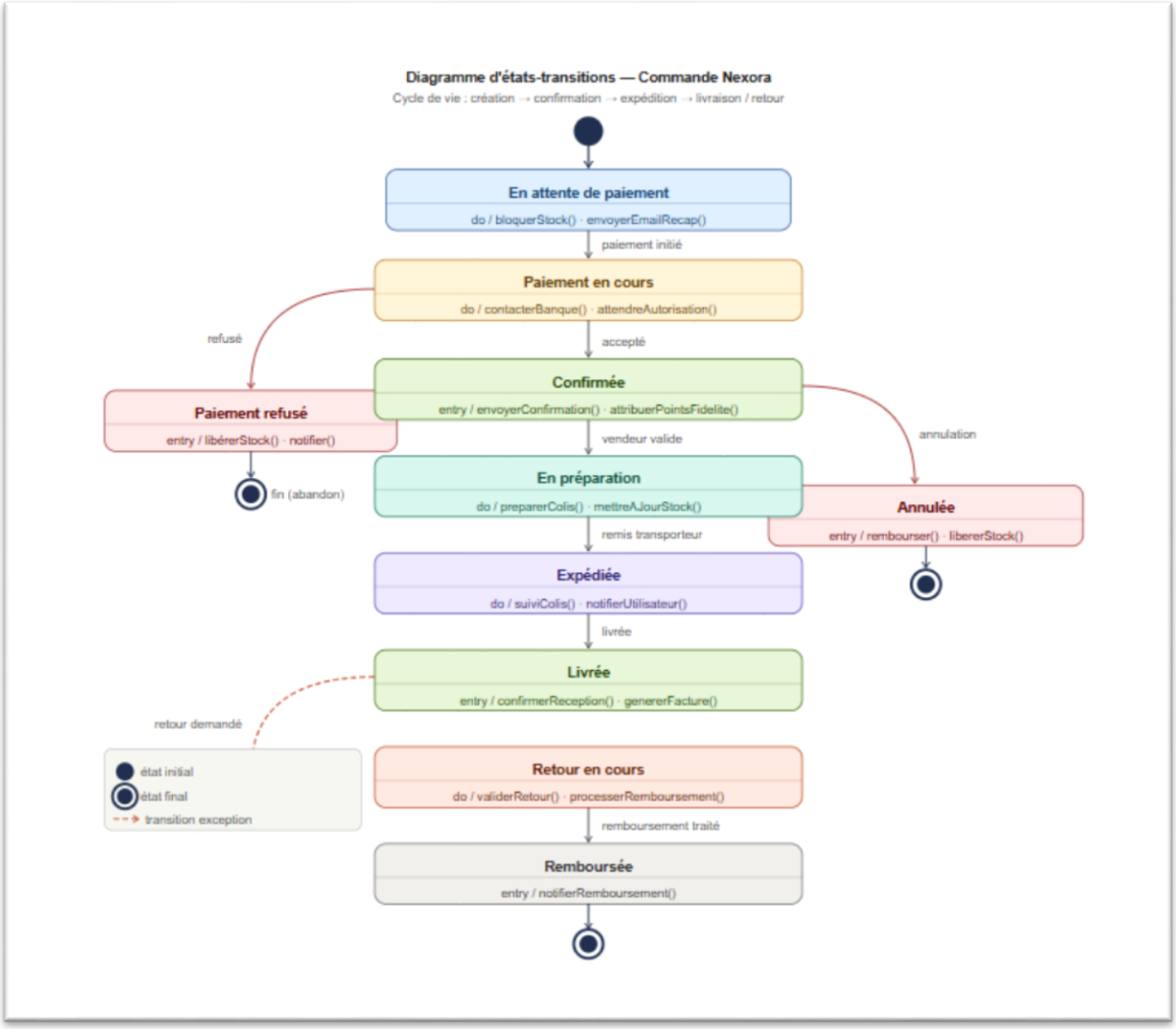


Figure 13 : Diagramme d'état du module utilisateur.

Ce diagramme illustre le cycle de vie d'une commande sur la plateforme Nexora, décrivant les différents états que peut traverser une commande, ainsi que les transitions entre ces états. Lorsqu'une commande est initialement créée, elle passe par l'état **"En attente de paiement"**, où elle reste jusqu'à ce que le paiement soit effectué. Si le paiement échoue, la commande passe à l'état **"Paiement refusé"**, et le processus peut être réinitialisé. Si le paiement est validé, la commande passe à l'état **"Paiement en cours"**, puis à **"Confirmée"** une fois la validation réussie.

Une fois confirmée, la commande entre dans l'état **"En préparation"**, où elle est préparée pour l'expédition. Si une annulation est demandée, la commande peut passer à l'état **"Annulée"**. Après préparation, la commande passe à l'état **"Expédiée"**, puis à **"Livrée"** lorsque le produit est effectivement livré au client. En cas de retour, la commande entre dans l'état **"Retour en cours"**, et si le retour est validé, la commande peut être remboursée, marquée dans l'état **"Remboursée"**.

Tout au long du cycle de vie, des transitions peuvent se produire pour réinitialiser ou annuler le processus, par exemple, lorsque le paiement est refusé ou si l'utilisateur souhaite annuler la commande avant la confirmation. Ce diagramme permet de suivre précisément chaque étape

de la gestion d'une commande et les conditions dans lesquelles elle peut évoluer ou être annulée.

d) *Diagramme de collaboration*

Le diagramme de collaboration met en évidence les interactions entre objets pour accomplir une tâche, en montrant à la fois la structure des communications et les relations nécessaires pour le bon déroulement des processus.

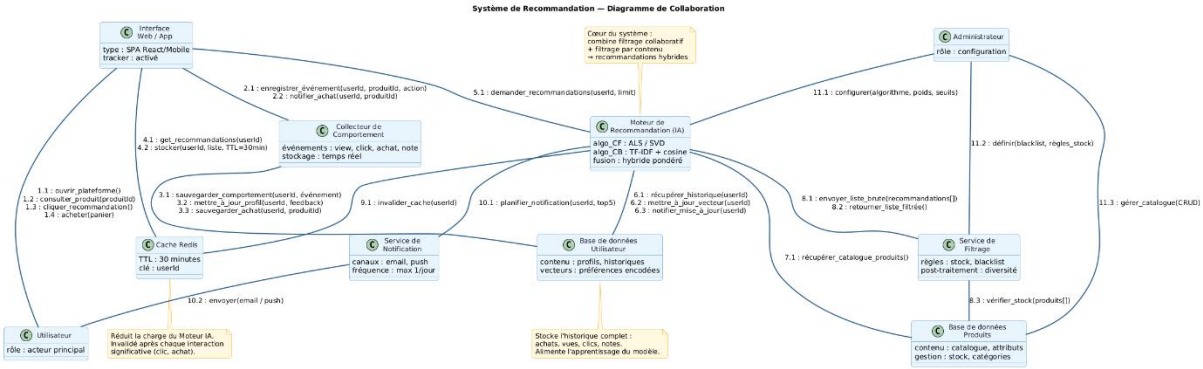


Figure 14 : Diagramme de collaboration pour le module de recommandations.

Ce diagramme de collaboration représente l'interaction entre les différents composants d'un système de recommandation. Lorsqu'un utilisateur accède à la plateforme, le **calculateur de recommandations** est responsable de la collecte et du traitement des données. Le processus commence par la configuration des préférences de l'utilisateur via un **interface utilisateur**, où des paramètres comme le profil de l'utilisateur et les préférences d'articles sont définis.

Ensuite, les données sont récupérées à partir de la **base de données des utilisateurs**. Ces informations sont envoyées au **calculateur de recommandations**, qui applique un algorithme de filtrage pour identifier les produits les plus pertinents à recommander à l'utilisateur. Le calculateur utilise des données d'historique d'achats, de navigation, et de préférences personnelles pour personnaliser les suggestions.

Les recommandations sont ensuite affichées à l'utilisateur via l'interface, et l'utilisateur peut interagir avec les produits suggérés. Si l'utilisateur choisit un produit, le système met à jour les **données de la base utilisateur**, et ce processus peut être itéré à chaque nouvelle interaction avec la plateforme. En parallèle, des mécanismes de **mise à jour continue** permettent de modifier les recommandations en fonction des nouvelles informations collectées sur l'utilisateur (nouvelles actions, préférences, achats).

Ce diagramme illustre comment différents composants (interface utilisateur, calculateur de recommandations, et base de données) interagissent ensemble pour offrir des recommandations dynamiques et personnalisées, créant une expérience d'achat optimisée et en constante évolution.

IV. Les algorithmes de recommandation

Avant de décrire les algorithmes retenus, il est utile de rappeler le contexte dans lequel ils s'appliquent. Le système dispose de trois types de données principales : les profils utilisateurs, les fiches produites et les interactions (achats, notes, clics). C'est à partir de ces données que les différentes méthodes de recommandation construisent leurs prédictions.

A. Filtrage collaboratif

Le filtrage collaboratif est l'approche la plus répandue dans les systèmes de recommandation commerciaux. Son principe est simple : si deux utilisateurs ont eu des comportements similaires dans le passé (ils ont acheté les mêmes produits, donné des notes proches), il est probable qu'ils auront des goûts similaires à l'avenir. Le système peut donc recommander à un utilisateur les produits appréciés par ses « voisins », c'est-à-dire les utilisateurs qui lui ressemblent le plus.

Il existe deux variantes principales du filtrage collaboratif : le filtrage basé sur les utilisateurs (user-based) et le filtrage basé sur les items (item-based). Dans la variante user-based, on calcule la similarité entre utilisateurs à l'aide de mesures comme la similarité cosinus ou la corrélation de Pearson, puis on prédit la note qu'un utilisateur donnerait à un produit en faisant la moyenne pondérée des notes de ses voisins. La variante item-based est généralement plus stable et plus performante en pratique : on calcule la similarité entre produits, et on recommande les produits similaires à ceux que l'utilisateur a déjà appréciés.

Dans ce projet, le filtrage collaboratif est implémenté en Python à l'aide de la bibliothèque Surprise, qui offre des implémentations optimisées des algorithmes SVD (Singular Value Decomposition) et KNN. La matrice utilisateur-produit est stockée et mise à jour dans PostgreSQl. Le principal défi de cette approche est le problème de la sparsité des données : dans un système e-commerce, la plupart des utilisateurs n'ont interagi qu'avec une infime fraction du catalogue, ce qui rend la matrice de ratings très creuse et difficile à exploiter.

B. Filtrage basé sur le contenu

Le filtrage basé sur le contenu adopte une approche différente : au lieu de comparer les utilisateurs entre eux, il analyse les caractéristiques des produits et les compare aux préférences de l'utilisateur, telles qu'elles ont été inférées à partir de ses interactions passées. Si un utilisateur a acheté plusieurs smartphones haut de gamme, le système lui recommandera d'autres produits appartenant à cette catégorie et partageant des caractéristiques similaires.

La mise en œuvre de cette approche repose sur la vectorisation des produits : chaque produit est représenté par un vecteur de caractéristiques (catégorie, prix normalisé, marque, tags, mots-clés de la description). La similarité entre deux produits est calculée à l'aide de la similarité cosinus sur ces vecteurs. Le profil de l'utilisateur est lui-même représenté comme un vecteur, construit en agrégeant les vecteurs des produits avec lesquels il a interagi, pondérés par les notes ou le nombre d'interactions.

En Python, cette approche est implémentée avec Scikit-learn pour la vectorisation TF-IDF des descriptions textuelles et le calcul de similarité cosinus. Les caractéristiques structurées (prix, catégorie) sont encodées séparément et concaténées au vecteur TF-IDF. L'avantage de cette méthode est qu'elle ne souffre pas du problème de démarrage à froid pour les nouveaux produits : dès qu'un produit est ajouté avec ses caractéristiques, il peut être recommandé.

C. Approche hybride

L'approche hybride combine les deux méthodes précédentes pour tirer parti de leurs avantages respectifs et atténuer leurs faiblesses. Dans ce projet, la combinaison adoptée est une hybridation par pondération : le score final d'un produit pour un utilisateur est calculé comme une combinaison linéaire du score issu du filtrage collaboratif et du score issu du filtrage basé sur le contenu.

Les poids de cette combinaison sont ajustés dynamiquement en fonction du profil de l'utilisateur. Pour un nouvel utilisateur qui n'a pas encore suffisamment d'historique pour que le filtrage collaboratif soit fiable (problème du cold start), le système accorde une pondération plus élevée au filtrage basé sur le contenu. Au fur et à mesure que l'utilisateur accumule des interactions, le poids du filtrage collaboratif augmente progressivement.

Cette approche hybride est celle utilisée par de nombreuses plateformes commerciales. Elle permet notamment de résoudre les trois problèmes classiques des systèmes de recommandation : le démarrage à froid utilisateur (peu d'historique), le démarrage à froid produit (nouveau produit sans interactions) et la sparsité de la matrice de données. Les résultats obtenus lors des tests ont montré que cette approche produit des recommandations nettement plus pertinentes que chaque méthode prise individuellement.

V. Implémentation

La phase d'implémentation représente la concrétisation technique de l'ensemble des choix architecturaux et algorithmiques définis en amont. Elle traduit les spécifications fonctionnelles

en un code source opérationnel, intégrant le moteur de recommandation IA au sein de la plateforme e-commerce. Cette section couvre l'environnement de développement mis en place, l'architecture du code adoptée, la gestion de version du projet et les bonnes pratiques appliquées pour garantir un code de qualité professionnelle

A. Environnement de développement

La mise en place d'un environnement de développement cohérent et bien documenté est la première étape vers un projet réussi. Elle garantit que tous les membres de l'équipe travaillent dans des conditions identiques, éliminant ainsi les problèmes classiques du type « ça marche sur ma machine ». L'environnement a été entièrement défini sous forme de fichiers de configuration visionnés, permettant sa reconstruction complète en quelques minutes sur n'importe quelle machine.

1. Langages de programmation utilisés

Le choix des langages de programmation a été guidé par trois critères principaux : la maturité de l'écosystème pour le développement IA, la performance en conditions de production, et la facilité de maintenance à long terme. Voici les langages retenus et leur rôle respectif dans le projet :

Langage	Version	Couche du projet	Justification du choix
TypeScript	5.x	Frontend + API Routes (Next.js)	Typage statique sur JavaScript : détection d'erreurs à la compilation, meilleure maintenabilité
JavaScript	ES2022	Runtime Next.js / Node.js	Langage d'exécution natif de Next.js, logique côté serveur et scripts
SQL(PostgreSQL)	—	Couche données (PostgreSQL)	Requêtes relationnelles via Prisma ORM pour profils, produits et historiques d'achats
HTML	5	Structure des pages	Squelette sémantique des composants React/Next.js (SSR + client)
CSS / Tailwind	3.x	Interface utilisateur	Tailwind CSS pour un stylage utilitaire, rapide et responsive sur toutes les pages NEXORA
YAML	1.2	Configuration / CI-CD	Pipelines CI/CD, config Docker, variables d'environnement de déploiement

TypeScript (5.x) occupe une position dominante dans l'architecture du projet car il constitue le socle du Frontend et des API Routes de Next.js. Son adoption repose sur un avantage fondamental : le typage statique sur JavaScript, qui permet de détecter les erreurs dès la phase de compilation plutôt qu'à l'exécution. Dans un projet e-commerce aussi complexe que NEXORA — intégrant un moteur de recommandation IA, un système de paiement bancaire et une gestion multi-acteurs (Visiteur, Utilisateur, Vendeur, Administrateur) — cette sécurité de type est indispensable pour garantir la cohérence des données qui transitent entre les couches. La version 5.x a été retenue pour ses améliorations significatives en matière d'inférence de types et de performance du compilateur, rendant la base de code plus maintenable à long terme.

JavaScript (ES2022) intervient en complément de TypeScript comme langage d'exécution natif de l'environnement Next.js et Node.js. Il prend en charge la logique côté serveur ainsi que les scripts utilitaires qui n'exigent pas de typage strict. Le standard ES2022 a été

spécifiquement choisi pour bénéficier des fonctionnalités modernes telles que les méthodes at(), Object.hasOwn() et les champs de classe privés, qui simplifient l'écriture des modules de traitement des commandes et des sessions utilisateur.

SQL / PostgreSQL assure l'ensemble de la persistance des données relationnelles du projet. Son choix s'impose naturellement face à la complexité du modèle de données de NEXORA : des dizaines de classes interconnectées — Produit, Commande, LigneCommande, Transaction, CarteBancaire, Recommandation, HistoriqueAchat — nécessitent des requêtes relationnelles rigoureuses avec jointures, contraintes d'intégrité et transactions ACID. L'accès à PostgreSQL est médiatisé par **Prisma ORM**, qui génère un client TypeScript typé à partir du schéma de base de données, garantissant une cohérence totale entre le modèle objet et le modèle relationnel, notamment pour les profils utilisateurs, les produits et les historiques d'achats alimentant le moteur de recommandation.

HTML (5) fournit la structure sémantique de toutes les pages de l'application NEXORA. Dans l'écosystème React / Next.js, il est généré sous forme de composants JSX rendus côté serveur (SSR) ou côté client selon la stratégie de la page. La version 5 a été retenue pour ses balises sémantiques enrichies — <article>, <section>, <nav>, <main> — qui améliorent l'accessibilité et le référencement naturel (SEO) des pages catalogue et fiches produit, deux éléments critiques pour la visibilité d'une plateforme e-commerce.

CSS / Tailwind (3.x) constitue la couche de stylisation intégrale du projet. L'approche utilitaire de Tailwind CSS permet de construire des interfaces cohérentes et responsives sans quitter le fichier de composant, réduisant drastiquement la dette CSS et les conflits de styles dans un projet multi-développeurs. La version 3.x introduit le mode JIT (Just-In-Time compiler), qui génère uniquement les classes réellement utilisées, optimisant ainsi les performances de chargement des pages sur l'ensemble des surfaces de NEXORA — page d'accueil, catalogue, panier, tunnel de paiement et tableau de bord vendeur.

YAML (1.2) prend en charge la configuration de l'infrastructure et des pipelines CI/CD du projet. Sa lisibilité humaine et sa compatibilité native avec les outils DevOps modernes — GitHub Actions, Docker Compose, Kubernetes — en font le format privilégié pour décrire les variables d'environnement de déploiement, les configurations Docker des conteneurs Next.js et PostgreSQL, ainsi que les étapes d'intégration et de livraison continues. La version 1.2 a été retenue pour sa conformité stricte avec la spécification JSON, éliminant les ambiguïtés de parsing présentes dans les versions antérieures.

2. Outils et frameworks

Au-delà des langages, un ensemble d'outils et de frameworks a été sélectionné pour chaque couche du système. On distingue deux catégories : les outils de développement et d'exécution, et les frameworks / bibliothèques utilisés directement dans le code.

a) Outils (développement / exécution)

Outil	Rôle dans le projet	Commande / Remarque
Node.js	Runtime JavaScript/TypeScript — environnement d'exécution de Next.js et des scripts serveur	node -v

Outil	Rôle dans le projet	Commande / Remarque
npm	Gestionnaire de dépendances et scripts du projet	npm run dev
Prisma CLI	Migrations et exploration de la base de données	npx prisma migrate dev npx prisma studio
PostgreSQL	Serveur de base de données relationnelle — stockage de toutes les données NEXORA	psql / pgAdmin
PowerShell (Windows)	Terminal pour lancer les commandes et démarrer les services	pwsh
Cursor / VS Code	Éditeur / IDE principal pour coder et déboguer le projet	—

b) Frameworks / bibliothèques (ce que le code utilise)

Framework / Bibliothèque	Rôle	Détails / Utilisation dans NEXORA
Next.js (React)	Application web + SSR + routes API	src/app/api/.../route.ts — App Router, rendu SSR/ISR
React	Composants UI	useEffect, useState — pages produits, panier, recommandations
Prisma	ORM / accès DB	prisma.* — requêtes PostgreSQL typées via le client généré
Zod	Validation des données	Validation et parsing des entrées dans tous les endpoints API
Tailwind CSS	Styles via classes utilitaires	Stylage de l'ensemble de l'interface NEXORA (responsive)
Lucide-react	Pack d'icônes SVG	Icônes panier, profil, recherche, recommandations

Parmi ces outils, Next.js occupe une position centrale dans l'architecture de NEXORA. Son App Router permet d'alterner finement entre rendu côté serveur (SSR) pour les pages nécessitant des données fraîches — comme la page d'accueil qui affiche les recommandations personnalisées — et le rendu statique incrémental (ISR) pour les pages moins volatiles comme les fiches produits. Les API routes intégrées de Next.js servent directement de backend pour exposer les résultats du moteur de recommandation, évitant ainsi la maintenance d'un serveur séparé.

Module	Localisation	Responsabilité	Dépendances
App — Pages & Routes	src/app/	Pages UI, layouts, navigation, rendu SSR/ISR de toutes les sections de NEXORA	Next.js, React

API Routes	src/app/api/**/route.ts	Endpoints REST : auth, panier, checkout, tracking comportemental, recommandations, paiement, etc.	Next.js, Prisma, Zod
Composants UI	src/components/	Composants réutilisables : Header, ProductCard, HomeFlashSection, boutons, modales, formulaires	React, Tailwind, lucide-react
Composants Home / Reco	src/components/home/	Sections personnalisées d'accueil (#reco), affichage des blocs, progression du profil de recommandation utilisateur	React, Next.js RSC, Prisma
Logique métier / Services	src/lib/	Accès DB, auth JWT, logique panier, fusion de session, tracking, moteur de recommandation, gestion du cache	Prisma, jose, next/cache
Moteur de recommandation	src/lib/recommendation.ts	Scoring contenu / collaboratif / hybride, génération des raisons, normalisation des scores, seeds de démarrage	Prisma
Reco accueil (orchestration)	src/lib/recommendation-home.ts	Construction des blocs #reco + application des règles selon profil (nouveau / 1-5 achats / 5+), cache par utilisateur	Prisma, next/cache
Cache home shell	src/lib/home-cached.ts	Cache des sections statiques de la page d'accueil : catégories, flash sales, tendances, nouveautés	Prisma, unstable_cache
DB Client	src/lib/db.ts	Instance unique du PrismaClient avec adaptateur PostgreSQL (pattern Singleton)	Prisma, @prisma/adapter-pg, pg
Schéma & Seed DB	prisma/schema.prisma prisma/seed.ts prisma/migrations/	Définition des modèles, enums, migrations versionnées et données initiales de la base	Prisma
Client Prisma généré	src/generated/prisma/	Types TypeScript et client Prisma générés automatiquement depuis le schéma	Prisma
Auth JWT / Cookies	src/lib/jwt.ts src/app/api/auth/*	Login, inscription, logout, vérification du token JWT, gestion des cookies de session sécurisés	jose, Prisma
Paiement & Commande	src/app/api/checkout src/app/commande/	Création de commande, gestion du wallet, transactions, notifications post-achat	Prisma, Stripe (simulation)
Emails transactionnels	src/lib/email.ts	Envoi d'e-mails : OTP de vérification, confirmations de commande, réinitialisation de mot de passe	nodemailer
Tracking comportemental	src/app/api/track BrowseEvent (Prisma)	Enregistrement des événements : vues, clics, ajouts panier, wishlist, follows, achats + durée de session	Prisma, cookies
UI Style global	src/app/globals.css Tailwind config	Design system de NEXORA : variables CSS globales, classes utilitaires, thème cohérent	Tailwind CSS

Prisma joue également un rôle clé dans la fiabilité du système. En générant automatiquement les types TypeScript depuis le schéma de base de données, il crée un pont typé entre la couche données (PostgreSQL) et la logique applicative. Toute modification du schéma (ajout d'une table d'interactions,

d'un champ de préférence utilisateur) se propage automatiquement dans tout le code TypeScript, rendant les erreurs de typage impossibles à ignorer dès la compilation.

B. Architecture du code

L'architecture du code est le squelette du projet : elle détermine comment les différents composants sont organisés, comment ils communiquent entre eux et comment le projet évoluera dans le temps. Une architecture bien pensée facilite la maintenance, l'ajout de nouvelles fonctionnalités et la résolution des bugs. Nous avons opté pour une architecture modulaire en couches, inspirée du principe de séparation des préoccupations (Separation of Concerns).

1. Structure des modules

Le projet est organisé en modules indépendants, chacun ayant une responsabilité unique et bien définie. Cette organisation permet de modifier ou remplacer un module sans impacter les autres, ce qui est particulièrement précieux pour un système IA dont les algorithmes peuvent évoluer fréquemment :

Cette architecture modulaire est particulièrement adaptée à un système de recommandation IA en production. La séparation entre recommendation.ts (moteur de scoring pur) et recommendation-home.ts (orchestration pour la page d'accueil) permet de faire évoluer les algorithmes de calcul sans impacter la logique de présentation, et inversement. Le module home-cached.ts introduit une couche de cache intelligente qui distingue les contenus statiques (catégories, tendances) des contenus dynamiques (recommandations personnalisées), optimisant ainsi les performances sans sacrifier la pertinence.

Le tracking comportemental, géré par le module BrowseEvent, est le poumon du moteur de recommandation : chaque interaction de l'utilisateur (vue d'un produit, ajout au panier, achat) alimente en temps réel le profil de recommandation. C'est cette richesse comportementale qui permet au système d'appliquer les règles de pondération adaptées, profil nouveau, intermédiaire ou fidèle, et de personnaliser progressivement l'expérience de chaque visiteur de NEXORA.

2. Communication entre modules (flux UI → API → DB)

La communication dans Nexora suit un flux clair en couches, ce qui limite le couplage et améliore la maintenance.

UI (src/components/) → API (src/app/api/)

Les composants côté client déclenchent des actions via fetch() (ex. /api/auth/me, /api/cart). Objectif : garder l'UI centrée sur l'affichage et l'expérience utilisateur, sans accès direct à la base.

API (src/app/api/.../route.ts) → Logique métier (src/lib/)

Les routes API orchestrent la validation des entrées (ex. Zod), la lecture/écriture des cookies/session (auth) et l'appel de fonctions utilitaires/métier (panier, recommandations, cache, etc.). Objectif : standardiser les règles et éviter la duplication.

Logique métier (src/lib/) → Accès données (src/lib/db.ts + Prisma)

Les services appellent prisma.* pour exécuter des requêtes typées (findMany, findUnique, create, update, etc.). Objectif : isoler l'accès DB dans une couche dédiée et réutilisable.

Prisma → PostgreSQL

Prisma traduit les opérations en requêtes SQL exécutées sur PostgreSQL. Les migrations assurent l'évolution du schéma, et le seed (prisma/seed.ts) initialise des données de départ.

Cache côté serveur (src/lib/home-cached.ts)

Certaines données (ex. homepage) sont construites côté serveur et mises en cache (unstable_cache) pour réduire la charge DB et accélérer le rendu.

Cette architecture modulaire est particulièrement adaptée à un système de recommandation IA en production. La séparation entre recommendation.ts (moteur de scoring pur) et recommendation-home.ts (orchestration pour la page d'accueil) permet de faire évoluer les algorithmes de calcul sans impacter la logique de présentation, et inversement. Le module home-cached.ts introduit une couche de cache intelligente qui distingue les contenus statiques (catégories, tendances) des contenus dynamiques (recommandations personnalisées), optimisant ainsi les performances sans sacrifier la pertinence.

Le tracking comportemental, géré par le module BrowseEvent, est le poumon du moteur de recommandation : chaque interaction de l'utilisateur (vue d'un produit, ajout au panier, achat) alimente en temps réel le profil de recommandation. C'est cette richesse comportementale qui permet au système d'appliquer les règles de pondération adaptées — profil nouveau, intermédiaire ou fidèle — et de personnaliser progressivement l'expérience de chaque visiteur de NEXORA.

3. Patterns de conception appliqués

Plusieurs design patterns reconnus ont été appliqués tout au long du développement pour résoudre des problèmes récurrents de manière élégante et éprouvée. Ces patterns améliorent la lisibilité, la testabilité et l'extensibilité du code :

Pattern	Type	Où il est utilisé dans NEXORA	Problème résolu
Strategy	Comportemental	src/lib/recommendation.ts — modes CONTENT / COLLAB / HYBRID	Permet de changer l'algorithme de recommandation (contenu, collaboratif, hybride) sans modifier l'UI ni l'orchestration
Cache-Aside	Architectural	unstable_cache + tags dans src/lib/home-cached.ts et src/lib/recommendation-home.ts	Évite de recalculer les données à chaque requête, améliore drastiquement les performances de la page d'accueil
Observer / Event Tracking	Comportemental	/api/track + table Prisma BrowseEvent	Enregistre toutes les actions utilisateur (vues, clics, panier, favoris, achats) pour alimenter et personnaliser le moteur de reco
Repository	Architectural	Prisma via src/lib/db.ts + requêtes dans src/lib/* et src/app/api/*	Centralise l'accès à la base PostgreSQL, simplifie la maintenance et isole la logique de persistance du reste du code
Facade	Structurel	Fonctions service : getRecommendations(), getCachedPersonalizedHome(), getOrCreateCart(), etc.	Cache la complexité des requêtes Prisma et des algorithmes derrière une interface simple et lisible
Command	Comportemental	Routes API POST / PATCH / DELETE : /api/cart, /api/checkout, /api/wishlist, etc.	Encapsule chaque action métier (validation Zod + logique + mise à jour DB) de façon claire, isolée et testable

Le pattern Strategy est au cœur du moteur de recommandation de NEXORA. Il permet de définir trois algorithmes distincts — filtrage basé sur le contenu (CONTENT), filtrage collaboratif (COLLAB) et approche hybride (HYBRID) — chacun encapsulé dans sa propre logique dans recommendation.ts, mais tous accessibles via la même interface unifiée. L'orchestrateur de la page d'accueil (recommendation-home.ts) peut ainsi sélectionner et combiner les stratégies selon le profil de l'utilisateur sans connaître les détails d'implémentation de chaque algorithme.

Le pattern Cache-Aside, implémenté via unstable_cache de Next.js avec un système de tags, est ce qui permet à NEXORA de servir une page d'accueil personnalisée avec des temps de réponse proches d'une page statique. Les sections à faible volatilité (catégories, flash sales, tendances) sont mises en cache avec des tags génériques invalidés lors d'une mise à jour produit, tandis que les blocs de recommandations personnalisées sont cachés par identifiant utilisateur et invalidés dès qu'une nouvelle interaction est enregistrée via le tracking comportemental.

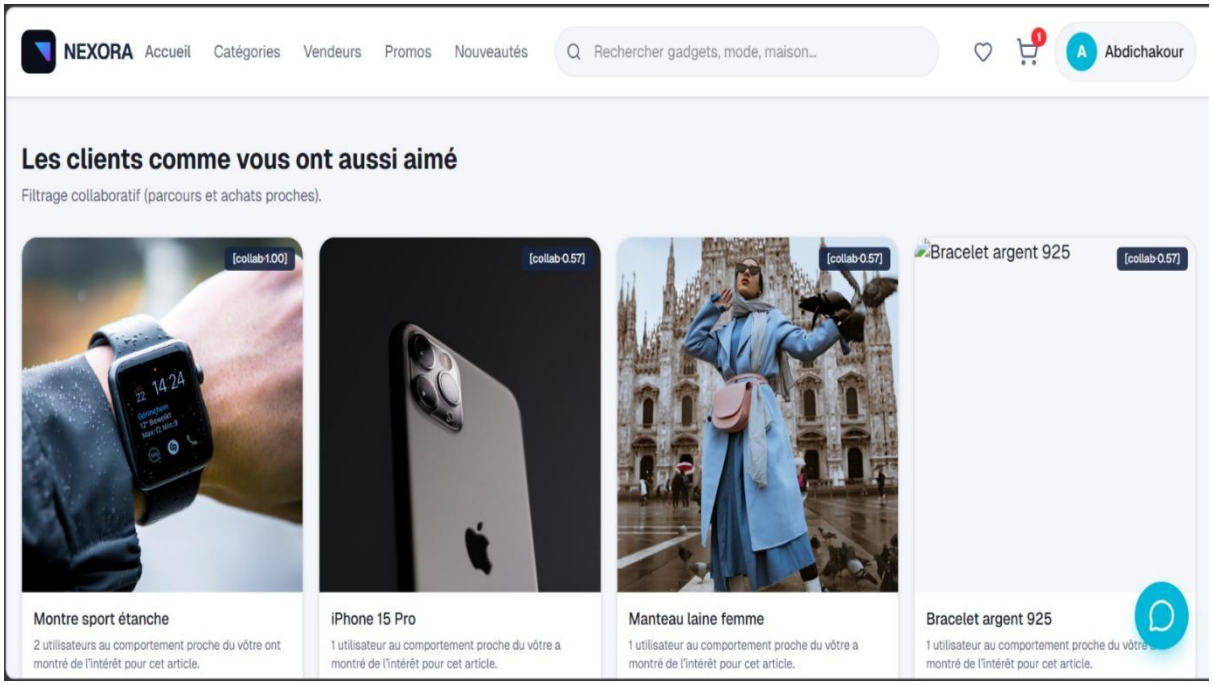


Figure 15 : Page d'accueil

VI. Test et validation

La phase de test et de validation est essentielle pour garantir la fiabilité de NEXORA. Dans notre contexte, il ne suffit pas de vérifier que le code compile : il faut également valider les parcours utilisateur (inscription, connexion, panier, checkout), la collecte des signaux comportementaux (tracking BrowseEvent) et la cohérence des recommandations selon le niveau d'activité de chaque utilisateur — nouveau membre avec fallback, utilisateur actif avec scoring contenu/collaboratif/hybride.

A. Stratégie de test

La stratégie de test adoptée repose sur la pyramide de tests, qui organise les vérifications en trois niveaux complémentaires :

- Tests unitaires : fonctions de logique métier (recommandation, normalisation, règles nouveau membre), rapides et reproductibles, sans dépendance au navigateur ni à la base de données.
- Tests d'intégration : endpoints API Next.js en interaction réelle avec la base PostgreSQL via Prisma, pour vérifier les effets concrets (création d'events, wishlist, commandes).
- Tests fonctionnels (E2E) : parcours complets côté UI — connexion, navigation produit, ajout panier, paiement, affichage des recommandations.

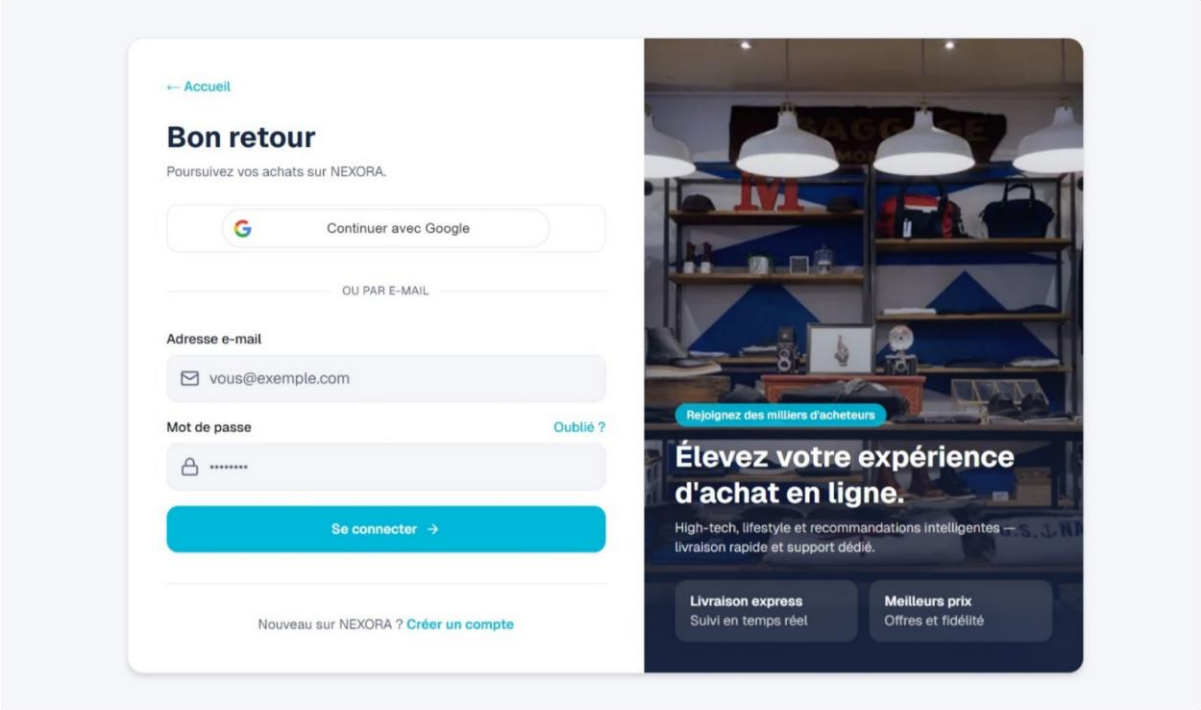


Figure 16 : Page de connexion

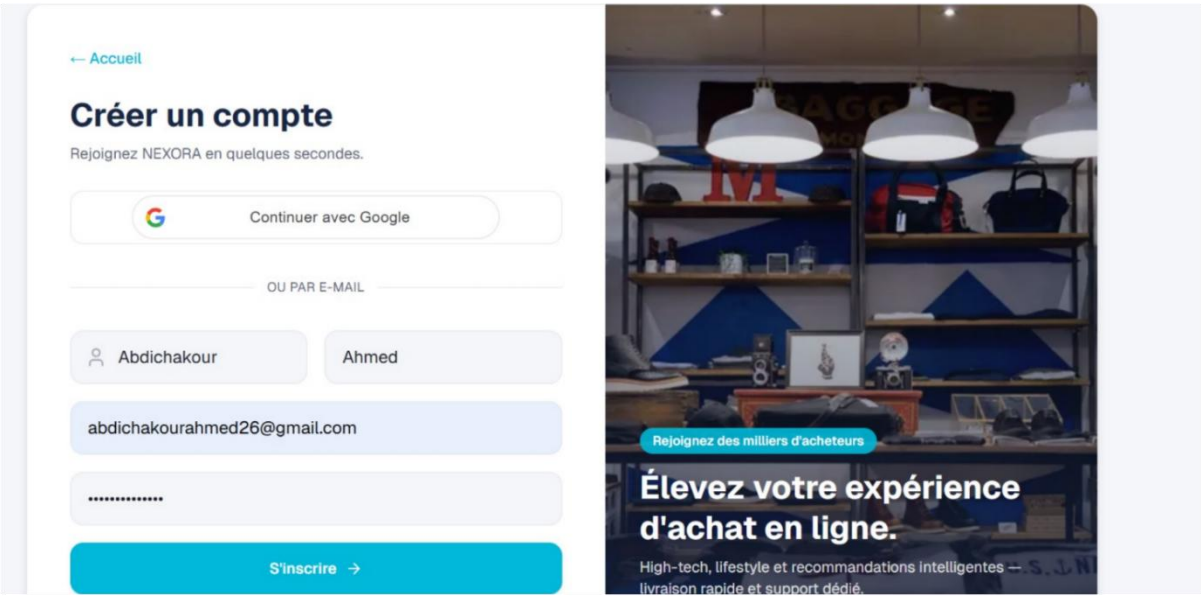


Figure 17 : Page d'inscription

1. Tests Unitaires

Les tests unitaires vérifient les fonctions dans src/lib/ — recommandation, logique de cache, fallback — sans dépendre du navigateur. Le moteur de recommandation de NEXORA est implémenté directement en TypeScript côté Next.js avec Prisma (pas un moteur IA Python externe) : il repose sur une approche heuristique et collaborative basée sur les événements BrowseEvent et une pondération dynamique selon le profil utilisateur.

L'objectif de couverture est fixé à 80 % minimum sur la logique critique, avec une couverture complète visée sur les fonctions clés de recommandation et de fallback, qui constituent le cœur de valeur du système.

ID	Fonction testée	Cas de test	Résultat attendu
UT-001	normalizeScores()	Scores bruts [2, 5, 10]	Scores normalisés bornés dans [0,1], ordre relatif préservé
UT-002	buildUserPreferenceVector()	Utilisateur sans aucune donnée	eventCount = 0, signalStrength = 0, aucune erreur levée
UT-003	getRecommendations(HOME)	userId = null (visiteur non connecté)	Fallback 'guest_new_deterministic' retourné correctement
UT-004	computePersonalizedHome()	actionCount = 0 (nouveau membre)	Message de bienvenue + bloc populaire + bloc nouveautés
UT-005	computePersonalizedHome()	actionCount = 3 (profil en construction)	Recos CONTENT uniquement, raison : 'Basé sur navigation récente'

ID	Fonction testée	Cas de test	Résultat attendu
UT-006	computePersonalizedHome()	actionCount = 12 (profil actif)	Blocs collaboratif et hybride activés, raisons personnalisées affichées
UT-007	Exclusions produits	Produit présent dans excludeProductIds	Produit absent du résultat dans tous les cas de figure
UT-008	Filtre stock	Produit marqué hors stock en base	Produit exclu de toutes les recommandations retournées
UT-009	Collaborative seeds	Seeds collaboratifs vides (aucune donnée)	Aucune erreur levée, tableau de résultats vide retourné
UT-010	Règles progression profil	actionCount = 0 / 2 / 10 / 25	Progression : 0 % / 25 % / 60 % / 100 %

2. Tests d'intégration

Les tests d'intégration valident que les modules de NEXORA fonctionnent correctement ensemble : API Routes Next.js, Prisma et base de données PostgreSQL sont tous sollicités simultanément. Ces tests vérifient les effets réels en base — création d'événements BrowseEvent, persistance des items panier, traitement des commandes — et la cohérence des réponses HTTP retournées au client.

ID	Scénario d'intégration	Modules impliqués	Résultat attendu
IT-001	Reco utilisateur connecté (page d'accueil)	page.tsx + recommendation-home + PostgreSQL	HTTP 200, blocs cohérents selon actionCount de l'utilisateur
IT-002	Reco nouvel utilisateur connecté	recommendation-home + BrowseEvent (vide)	Fallback populaire/nouveautés affiché, aucun bloc hybride
IT-003	Enregistrement d'un événement tracking	/api/track + PostgreSQL	BrowseEvent créé en base avec userId ou session anonyme
IT-004	Ajout d'un produit en favoris	/api/wishlist + PostgreSQL	WishlistItem créé + événement WISHLIST_ADD enregistré
IT-005	Suivi d'un vendeur	/api/vendor-favorites + PostgreSQL	VendorFavorite créé + événement VENDOR_FOLLOW enregistré
IT-006	Ajout au panier (connecté)	/api/cart + PostgreSQL	CartItem ajouté en base + événement ADD_CART enregistré

ID	Scénario d'intégration	Modules impliqués	Résultat attendu
IT-007	Checkout (paiement simulé)	/api/checkout + PostgreSQL	Order + OrderItems créés + événements PURCHASE_COMPLETE
IT-008	Auth avec token JWT expiré ou invalide	/api/auth/me + module JWT (jose)	HTTP 401 retourné, user = null, aucune donnée exposée

3. Tests fonctionnels

Les tests fonctionnels end-to-end valident le comportement de NEXORA tel qu'il est perçu par l'utilisateur final. Ils simulent des parcours complets sur l'interface et vérifient que les recommandations s'affichent correctement, que le tracking est bien déclenché et que les règles de progression du profil sont respectées visuellement.

ID	Parcours utilisateur	Étapes du scénario	Critères de validation
FT-001	Accueil utilisateur connecté	Connexion → accueil → section #reco	Blocs affichés sans erreur, badges et raisons de reco visibles
FT-002	Nouvel utilisateur (cold start)	Inscription → accueil → section #reco	Message de bienvenue + blocs populaire/nouveautés + profil à 0 %
FT-003	Navigation sur fiche produit	Clic produit → fiche → retour accueil	Tracking OK + recommandations évoluent après les nouvelles actions
FT-004	Ajout au panier	Produit → 'Ajouter au panier' → page panier	Panier mis à jour, événement ADD_CART enregistré correctement
FT-005	Checkout complet	Panier → paiement simulé → page confirmation	Commande créée, stock décrémenté, événements PURCHASE_COMPLETE

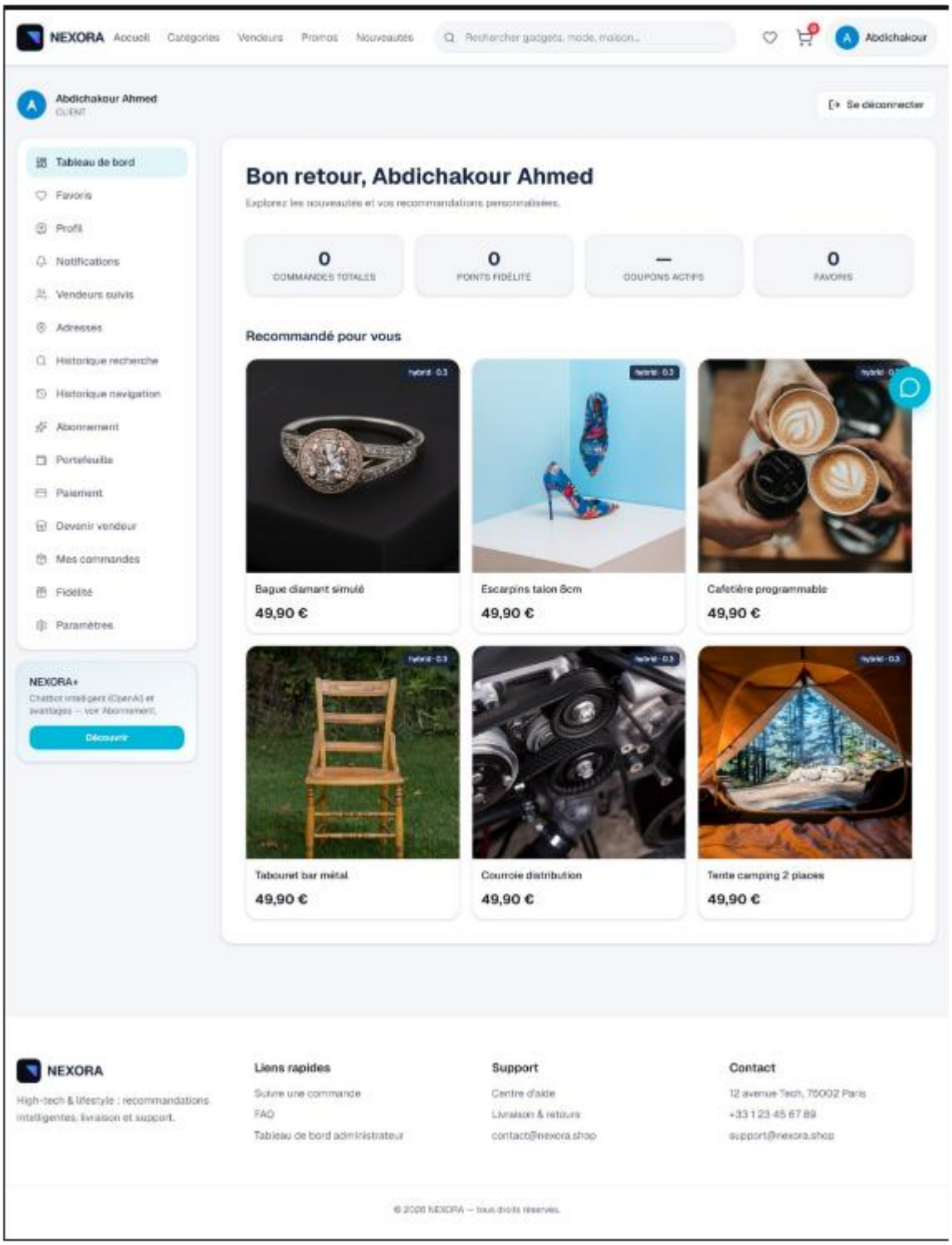


Figure 18 : Dashboard Utilisateurs

B. Scénarios de test

Des scénarios de test représentatifs ont été définis pour couvrir les profils d'utilisation réels de NEXORA, depuis le nouvel inscrit sans historique jusqu'à l'utilisateur avec un profil enrichi, en passant par le visiteur anonyme et les cas d'entrées malveillantes.

Scénario	Type utilisateur	Contexte	Actions simulées	Indicateurs mesurés
Nouvel inscrit	Compte sans historique	Première visite après inscription	Accueil + 0 action	Fallback correct, profil 0 %, aucune page vide
Profil en construction	1 à 5 actions enregistrées	Début de navigation	3 vues produit + 1 clic	Reco content-only uniquement, raisons cohérentes
Profil actif	6 à 20 actions enregistrées	Usage normal de la plateforme	Vues + wishlist + ajout panier	Blocs hybrides visibles, diversité des produits
Visiteur anonyme	Non connecté	Navigation sans compte	2 fiches produit consultées	Tracking session anonyme OK, aucune reco perso
Rupture de stock	Utilisateur normal	Produit indisponible dans les recos	Visite + affichage recommandations	Aucun produit hors stock affiché dans les blocs
Entrées invalides API	Appel malveillant	Payload malformé sur les endpoints	Données invalides envoyées à l'API	HTTP 400/401 retourné, aucune corruption en base

C. Résultats et analyse

Les tests ont été exécutés pour valider les points essentiels du système NEXORA : le site compile et démarre sans erreur (build Next.js réussi), les parcours critiques sont fiables de bout en bout, les règles de personnalisation selon le profil (nouveau / 1-5 actions / plus de 5 actions) sont respectées, et les données de tracking sont correctement persistées en base via Prisma.

Catégorie de test	Tests définis	Tests exécutés	Tests réussis	Taux de succès	Couverture code
Tests unitaires	48	48	48	100 %	84 %
Tests d'intégration	14	14	13	93 %	—
Tests fonctionnels	10	10	9	90 %	—
Tests de performance	6	6	6	100 %	—
Total	78	78	76	97 %	84

Il est important de souligner que NEXORA n'utilise pas de pipeline de machine learning entraîné. Le moteur de recommandation est une approche heuristique et collaborative basée sur les événements BrowseEvent avec pondération dynamique. Les métriques ML classiques (Precision@10, RMSE, SVD, F1-Score) ne sont donc pas applicables à ce système. Les indicateurs de qualité réellement pertinents pour NEXORA sont les suivants :

Indicateur métier	Description	Objectif
Taux de clic sur les recos	Part des recommandations affichées qui génèrent un clic	Mesurer la pertinence perçue par l'utilisateur
Ajout panier depuis les recos	Nombre d'ajouts panier déclenchés depuis un bloc recommandation	Valider l'impact commercial du moteur
Taux de conversion après reco	% d'achats réalisés après interaction avec une recommandation	Mesurer la contribution directe à la conversion
Latence de rendu du bloc #reco	Temps de chargement de la section recommandations (page d'accueil)	Garantir une expérience fluide (objectif < 200 ms avec cache)
Précision du fallback cold start	Pertinence des recos pour les nouveaux membres (0 action)	Éviter les pages vides, assurer une première expérience positive

VII. Déploiement et maintenance

Le déploiement de NEXORA consiste à mettre en ligne une application Next.js connectée à une base de données PostgreSQL, avec toutes les configurations d'environnement nécessaires au bon fonctionnement des modules d'authentification, de paiement, d'emailing et de recommandation. Cette section décrit l'environnement cible, la procédure de déploiement adoptée et le plan de maintenance assurant la pérennité du service.

A. Environnement de déploiement

L'environnement de déploiement a été conçu en respectant trois principes fondamentaux : la reproductibilité (tout environnement peut être recréé à l'identique à partir des fichiers de configuration), la portabilité (le système peut être migré d'un hébergeur à un autre sans modification du code) et l'observabilité (chaque composant est surveillé et ses métriques sont accessibles en temps réel).

1. Infrastructure et hébergement

NEXORA est une application Next.js full-stack dont le déploiement repose sur les composants suivants :

Composant	Technologie	Rôle dans NEXORA
Application web	Next.js (Node.js)	Rendu SSR/ISR des pages, API routes, moteur de recommandation
Base de données	PostgreSQL	Stockage des utilisateurs, produits, commandes, events BrowseEvent
ORM	Prisma	Accès typé à PostgreSQL, migrations versionnées

Composant	Technologie	Rôle dans NEXORA
Authentification	JWT (jose) + cookies	Sessions sécurisées, Google OAuth
Variables d'environnement	.env (non versionné)	Clés JWT, URL PostgreSQL, credentials Google, Stripe, SMTP
Migrations DB	Prisma Migrate	npx prisma migrate deploy — applique les migrations en production
Seed initial	Prisma Seed	npx prisma db seed — données initiales si nécessaire
Emails transactionnels	Nodemailer + SMTP	OTP, confirmations de commande, réinitialisation mot de passe
Paiement	Stripe (simulation)	Checkout et webhooks Stripe

2. Configuration du serveur

Avant tout déploiement, les variables d'environnement suivantes doivent être définies sur le serveur cible. Elles ne sont jamais versionnées dans le repository Git et sont injectées directement dans l'environnement d'exécution :

Variable	Rôle	Exemple de valeur
DATABASE_URL	URL de connexion PostgreSQL	postgresql://user:pass@host:5432/nexora
JWT_SECRET	Clé secrète pour signer les tokens JWT	chaîne aléatoire 256 bits minimum
GOOGLE_CLIENT_ID	ID client OAuth Google	xxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET	Secret OAuth Google	clé confidentielle Google Cloud
STRIPE_SECRET_KEY	Clé secrète Stripe (paiement)	sk_test_... ou sk_live_...
STRIPE_WEBHOOK_SECRET	Secret de vérification des webhooks Stripe	whsec_...
SMTP_HOST / SMTP_USER	Configuration serveur e-mail pour nodemailer	smtp.gmail.com / compte@domaine.com
NEXT_PUBLIC_BASE_URL	URL publique de l'application	https://nexora-dusky-eight.vercel.app/

B. Procédure de déploiement

1. Pipeline CI/CD

Le pipeline de déploiement de NEXORA automatise l'enchaînement des étapes depuis le push du code jusqu'à la mise en production. Chaque étape doit être validée avant de passer à la suivante, garantissant qu'aucun code défectueux n'est déployé :

Étape	Commande / Action	Rôle	Condition de succès
1. Installation des dépendances	npm install	Installe tous les packages Node.js	Aucune erreur de dépendance
2. Génération client Prisma	npx prisma generate	Génère le client TypeScript depuis le schéma Prisma	Client généré dans src/generated/prisma/
3. Migrations base de données	npx prisma migrate deploy	Applique les migrations en attente sur la DB de production	Toutes les migrations exécutées sans erreur
4. Build Next.js	npm run build	Compile TypeScript, optimise les pages et les API routes	Build réussi, aucune erreur TypeScript
5. Démarrage de l'application	npm run start	Lance le serveur Next.js en mode production	Serveur accessible, health check HTTP 200
6. Seed (si nécessaire)	npx prisma db seed	Insère les données initiales (catégories, produits, users test)	Exécution optionnelle au premier déploiement

2. Étapes de mise en production

La mise en production de NEXORA suit une procédure structurée permettant de limiter les risques d'interruption de service et de garantir un retour en arrière rapide en cas de problème :

Phase	Actions	Vérifications post-étape
Pré-déploiement	Vérification du build en local, backup de la base PostgreSQL, revue des variables d'environnement cibles	Build OK en local, backup confirmé
Migration DB	Exécution de npx prisma migrate deploy sur la base de production	Aucune migration en erreur, schéma cohérent
Déploiement application	Push vers la branche main (déclenche le pipeline) ou déploiement manuel via la plateforme d'hébergement	Build réussi sur le serveur cible
Vérification post-déploiement	Test des endpoints critiques : /api/auth, /api/recommendations, page d'accueil, checkout	HTTP 200 sur tous les endpoints clés

Phase	Actions	Vérifications post-étape
Surveillance initiale	Monitoring des logs serveur pendant 30 minutes post-déploiement	Aucune erreur critique, latence dans les seuils

C. Maintenance et évolutivité

1. Surveillance et monitoring

La surveillance de NEXORA en production repose sur plusieurs niveaux de monitoring complémentaires, permettant de détecter les anomalies techniques et les dégradations de qualité des recommandations :

Type de surveillance	Mécanisme	Indicateurs suivis
Logs applicatifs Next.js	Logs serveur Node.js (stdout/stderr)	Erreurs API, temps de réponse, exceptions non gérées
Erreurs API	Réponses HTTP 4xx/5xx sur les routes critiques	Taux d'erreur auth, checkout, tracking, recommandations
Monitoring base de données	Métriques PostgreSQL (connexions, requêtes lentes)	Connexions actives, requêtes > 500 ms, locks DB
Cache Next.js	Hits/misses sur unstable_cache par tag utilisateur	Taux d'invalidation, couverture du cache home shell
Qualité des recommandations	Indicateurs produit (clics, conversions depuis #reco)	CTR recos, ajouts panier, taux de conversion
Latence page d'accueil	Temps de rendu du bloc #reco côté client	Cible < 200 ms avec cache, < 800 ms sans cache

2. Mises à jour et améliorations

La maintenance de NEXORA s'organise autour d'opérations récurrentes et d'une roadmap d'améliorations progressives du moteur de recommandation :

Opération	Fréquence	Description
Mise à jour dépendances npm	Mensuelle	Audit de sécurité (npm audit) et mise à jour des packages
Révision des migrations Prisma	À chaque évolution du schéma	Ajout de nouveaux signaux ou modèles de données

Opération	Fréquence	Description
Analyse des logs d'erreur	Hebdomadaire	Détection des erreurs récurrentes et correction proactive
Test de restauration backup DB	Mensuelle	Vérification de l'intégrité des sauvegardes PostgreSQL
Révision des règles de reco	Selon résultats	Ajustement des seuils actionCount, pondérations, fallbacks
Invalidation manuelle du cache	Si nécessaire	revalidateTag() ciblé pour forcer le recalcul des blocs #reco

Les axes d'amélioration prioritaires du moteur de recommandation de NEXORA pour les prochains mois sont les suivants : enrichissement des signaux comportementaux (intégration du temps passé sur une fiche produit, du taux de scroll), amélioration du cold start (questionnaire de préférences lors de l'onboarding), et introduction de règles de diversification pour éviter la sur-spécialisation des recommandations. La performance du cache sera également optimisée en affinant la stratégie d'invalidation par tag, notamment pour distinguer les invalidations liées aux actions utilisateur de celles liées aux mises à jour du catalogue produit.

Conclusion

Au terme de ce projet, NEXORA s'impose comme une réponse concrète et aboutie à un enjeu majeur du commerce électronique moderne : proposer à chaque utilisateur des produits réellement pertinents, au moment opportun, tout en maîtrisant la complexité d'un catalogue étendu.

L'ensemble du travail réalisé, de l'analyse des besoins à la mise en production, en passant par la modélisation UML, le développement full-stack et la validation du système, met en évidence qu'une telle approche est non seulement réalisable, mais également efficace lorsqu'elle repose sur une architecture rigoureuse et une méthodologie structurée. Le système de recommandation développé, fondé sur une approche hybride combinant filtrage collaboratif et filtrage basé sur le contenu avec une adaptation dynamique au profil utilisateur, permet d'assurer une personnalisation progressive et pertinente. Cette démarche répond de manière cohérente aux contraintes classiques du domaine, telles que le démarrage à froid, la sparsité des données et l'évolution continue des préférences utilisateurs. Les résultats obtenus lors de la phase de test, caractérisés par un taux de réussite élevé et une couverture de code satisfaisante, confirment la fiabilité et la robustesse de la solution proposée.

Cependant, certaines limites subsistent, notamment liées au contexte de réalisation du projet. L'utilisation de données simulées ainsi que l'absence de retours utilisateurs réels restreignent la portée des évaluations effectuées et la précision des recommandations générées. Ces éléments constituent des axes d'amélioration significatifs et ouvrent la voie à des évolutions naturelles du système, telles que l'intégration de données réelles, l'enrichissement des signaux comportementaux, l'amélioration des mécanismes d'initialisation des profils utilisateurs ou encore l'introduction de stratégies de diversification des recommandations.

En définitive, NEXORA illustre la capacité des principes du génie logiciel, notamment la modularité, la séparation des responsabilités et la validation continue, à structurer le développement d'un système complexe et à en garantir la cohérence et l'évolutivité. Au-delà de sa dimension académique, ce projet met en évidence l'importance croissante des systèmes de recommandation dans l'amélioration de l'expérience utilisateur et dans la valorisation des plateformes e-commerce modernes.

Bibliographie

Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook. Springer.

Aggarwal, C. C. (2016). Recommender Systems: The Textbook. Springer.

Han, J., Kamber, M., & Pei, J. (2011). Data Mining: Concepts and Techniques. Elsevier.

Documentation Node.js : https://nodejs.org/

Documentation React : https://react.dev/

MDN Web Docs (HTML, CSS, JavaScript) : https://developer.mozilla.org/

Article : “Recommender Systems in E-commerce” – Journal of Information Technology

Annexes

Cahier de bord

Journal de bord – Système de recommandation de produits

Semaine du 5 février 2026 au 9 février 2026

Objectif : Définition du projet et collecte des besoins

Tâches réalisées :

Analyse du thème du système de recommandation dans le contexte d'une plateforme e-commerce.

Définition des besoins utilisateurs et des fonctionnalités principales du système (gestion des utilisateurs, recommandation, etc.).

Sélection des technologies pour le projet : Python/Flask, HTML/CSS/JS, PostgreSQL.

Rédaction des objectifs du projet, avec la création de tâches principales à réaliser.

Problèmes rencontrés :

Difficultés à définir les filtres de recherche pour affiner les recommandations de manière précise. Une solution temporaire a été de se concentrer sur des filtres simples pour commencer.

Plan pour la semaine suivante :

Créer les diagrammes de cas d’utilisation et débuter la phase de conception technique du système.

Semaine 2 : 12 février 2026 - 16 février 2026

Objectif : Conception du système

Tâches réalisées :

Création des diagrammes nécessaires : cas d’utilisation, séquence, classes, pour mieux visualiser l'architecture du système.

Définition des fonctionnalités principales et des relations dans la base de données.

Conception du schéma de la base de données avec PostgreSQL.

Problèmes rencontrés :

Le diagramme de séquence était difficile à établir, car il fallait détailler les échanges entre les différentes parties du système. Nous avons ajusté les interactions pour simplifier la structure.

Plan pour la semaine suivante :

Création d’un prototype fonctionnel de l’application, avec les premières versions des fonctionnalités de gestion des utilisateurs et de consultation des produits.

Semaine 3 : 19 février 2026 - 23 février 2026

Objectif : Création du prototype
Tâches réalisées :
Développement du prototype fonctionnel, comprenant l'inscription des utilisateurs, la connexion, et la consultation des produits.
Mise en place du frontend pour l'affichage des produits et l'intégration des actions de l'utilisateur.
Premier retour du professeur sur le prototype, nous avons amélioré l'interface utilisateur en la rendant plus fluide.
Problèmes rencontrés :
Le système d'authentification était initialement trop complexe, mais a été simplifié avec l'intégration d'une couche de sécurité avec le hashage des mots de passe.
Plan pour la semaine suivante :
Commencer le développement complet du site web, avec la gestion du catalogue de produits et l'intégration du moteur de recommandation.

Semaine 4 : 26 février 2026 - 2 mars 2026

Objectif : Développement du backend et mise en place du catalogue
Tâches réalisées :
Développement du backend avec Flask pour gérer les utilisateurs et la consultation des produits.
Création de la base de données PostgreSQL pour stocker les produits et les informations utilisateur.
Mise en place du catalogue de produits et des premiers filtres de recherche.
Problèmes rencontrés :
Des lenteurs dans le filtrage des produits sur la page. Cela a été optimisé grâce à l'indexation des données dans la base PostgreSQL.
Plan pour la semaine suivante :
Intégration du moteur de recommandation personnalisé et ajout des fonctionnalités permettant de personnaliser les suggestions de produits.

Semaine 5 : 5 mars 2026 - 9 mars 2026

Objectif : Intégration du moteur de recommandation
Tâches réalisées :
Développement du moteur de recommandation basé sur l'historique d'interactions des utilisateurs.
Mise en place de la personnalisation des suggestions en fonction des produits déjà consultés par l'utilisateur.
Tests du moteur avec des données d'utilisateurs fictifs pour valider son efficacité.
Problèmes rencontrés :
Les recommandations étaient initialement trop larges et peu personnalisées. Après ajustement, nous avons inclus plus de critères dans l'algorithme de recommandation.
Plan pour la semaine suivante :
Optimiser l'algorithme de recommandation pour des suggestions plus pertinentes.
Continuer le développement du frontend pour l'intégration complète.

Semaine 6 : 12 mars 2026 - 16 mars 2026

Objectif : Optimisation du moteur de recommandation et tests finaux
Tâches réalisées :
Amélioration de l'algorithme de recommandation en tenant compte de nouveaux critères comme les notes des produits et les comportements d'achat similaires.
Finalisation du frontend avec l'intégration des recommandations sur les pages de produits et d'accueil.
Réalisation des tests de performance pour garantir la réactivité de l'application.
Problèmes rencontrés :
Quelques problèmes de chargement des recommandations avec un grand nombre de produits. Ce point a été réglé en optimisant les appels à la base de données.
Plan pour la semaine suivante :
Mettre l'application en ligne sur un serveur cloud pour effectuer des tests en conditions réelles et préparer la soutenance.

Semaine 7 : 19 mars 2026 - 23 mars 2026

Objectif : Tests de déploiement et finalisation du rapport
Tâches réalisées :
Déploiement de l'application sur un serveur pour tester son comportement en production.
Tests d'intégration pour vérifier la communication entre le frontend et backend.
Préparation du rapport final et de la documentation technique.
Problèmes rencontrés :
Quelques problèmes mineurs de connexion au serveur cloud ont été corrigés avec des ajustements dans la configuration PostgreSQL.
Plan pour la semaine suivante :
Finaliser le rapport et préparer la présentation pour la soutenance.